

Data Structure: 트리

Prof. Jungmin Kwon
Spring, 2026

■ 학습목표

- 트리의 개념을 이해한다.
- 이진 트리의 자료구조를 알아본다.
- 이진 트리의 순회를 이해한다.
- 이진 탐색 트리의 개념을 이해하고 연산 방법을 이해한다.
- 힙의 자료구조를 이해한다.

■ 내용

- 01 트리의 이해
- 02 이진 트리
- 03 이진 트리의 구현
- 04 이진 트리의 순회
- 05 이진 탐색 트리
- ~~06 균형 이진 탐색 트리~~
- 07 힙의 개념과 연산 및 구현

1. 트리의 이해

■ 트리|tree

- 원소들 간에 1:n 관계를 가지는 비선형 자료구조
- 원소들 간에 계층관계를 가지는 계층형 자료구조 Hierarchical Data Structure
- 상위 원소에서 하위 원소로 내려가면서 확장되는 트리(나무)모양의 구조

1. 트리의 이해

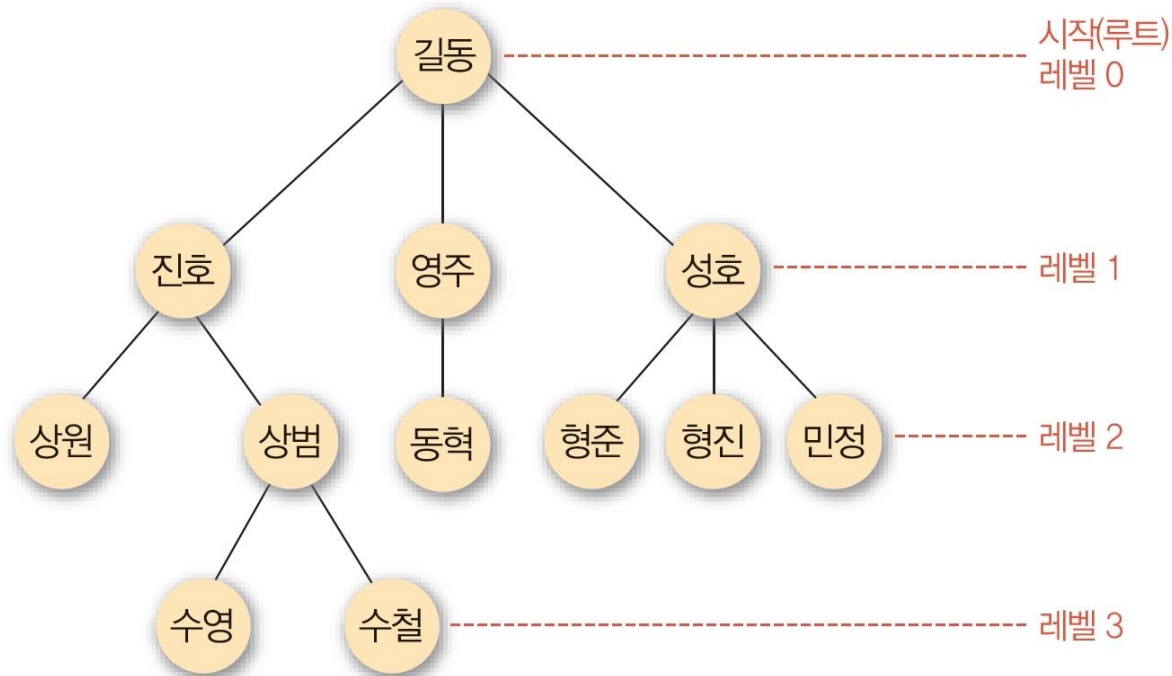


그림 7-1 트리 자료구조의 예 : 가계도

- 트리 자료구조의 예 - 가계도
 - 가계도의 자료 : 가족 구성원
 - 자료를 연결하는 선 : 부모-자식 관계 표현

1. 트리의 이해

- 트리 자료구조의 예 - 가계도 (이어서..)

- 길동의 자식 - 진호, 영주, 성호
- 진호, 영주, 성호의 부모 - 길동
- 같은 부모의 자식들끼리는 형제관계
 - 진호, 영주, 성호는 형제관계

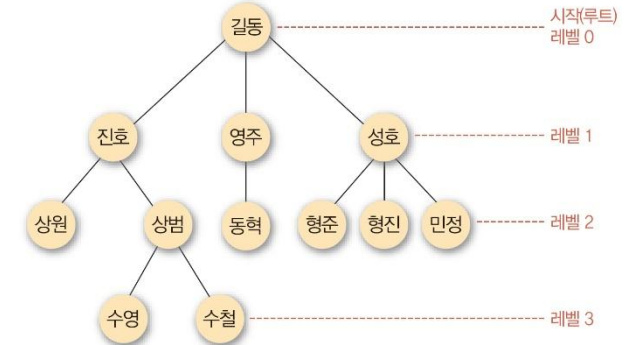


그림 7-1 트리 자료구조의 예 : 가계도

- 조상 - 자신과 연결된 선을 따라 올라가면서 만나는 사람들
 - 수영의 조상 : 상범, 진호, 길동
- 자손 - 자신과 연결된 선을 따라 내려가면서 만나는 사람들
 - 진호의 자손 : 상원, 상범, 수영, 수철
- 가계도의 시작(루트)인 길동을 0세대(레벨 0), 길동이 자식들은 1세대(레벨 1), 그 다음 자식들은 2세대(레벨 2), 그 다음 자식들은 3세대(레벨 3)
- 가족 구성원 누구든지 분가하여 독립된 가계를 이룰 수 있음

- 트리 A

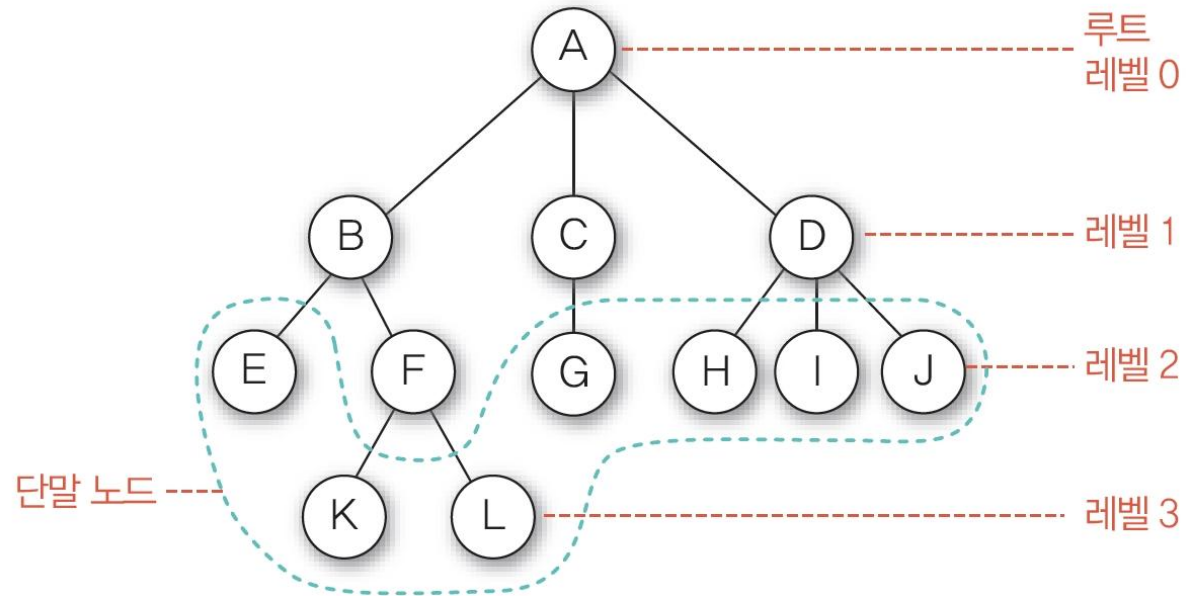


그림 7-2 트리의 구조와 구성 요소

1. 트리의 이해

- 트리 A

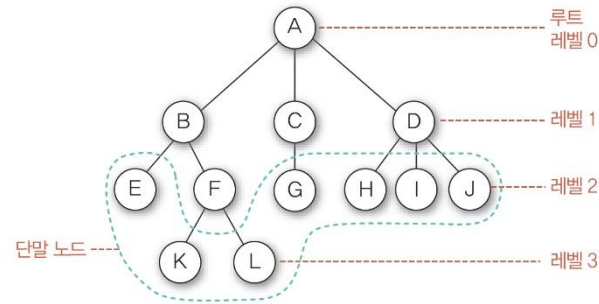


그림 7-2 트리의 구조와 구성 요소

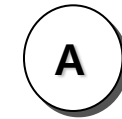
이름	설명	비고
노드	트리의 원소	트리 A의 노드 - A,B,C,D,E,F,G,H,I,J,K,L
루트 노드	트리의 시작 노드(레벨 0)	트리 A의 루트노드 - A
간선(edge)	노드를 연결하는 선. 부모 노드와 자식 노드를 연결	
형제 노드	같은 부모 노드의 자식 노드들	B,C,D는 형제 노드
조상 노드	간선을 따라 루트 노드까지 경로에 있는 모든 노드들	K의 조상 노드 : F, B, A
서브 트리	부모 노드와 연결된 간선을 끊었을 때 생성되는 트리	각 노드는 자식 노드의 개수 만큼 서브 트리를 가짐
자손 노드	서브 트리에 있는 하위 레벨의 노드들	B의 자손 노드 - E,F,K,L

1. 트리의 이해

– 트리 A

• 차수(degree)

- ▶ 노드의 차수 : 노드에 연결된 자식 노드의 수
→ A의 차수=3, B의 차수=2, C의 차수=1
- ▶ 트리의 차수 : 트리에 있는 노드의 차수 중에서 가장 큰 값
→ 트리 A의 차수=3
- ▶ 단말 노드(리프 노드) : 차수가 0인 노드. 자식 노드가 없는 노드



• 높이

- ▶ 노드의 높이 : 루트에서 노드에 이르는 간선의 수. 노드의 레벨
→ B의 높이=1, F의 높이=2
- ▶ 트리의 높이 : 트리에 있는 노드의 높이 중에서 가장 큰 값. 최대 레벨
→ 트리 A의 높이=3

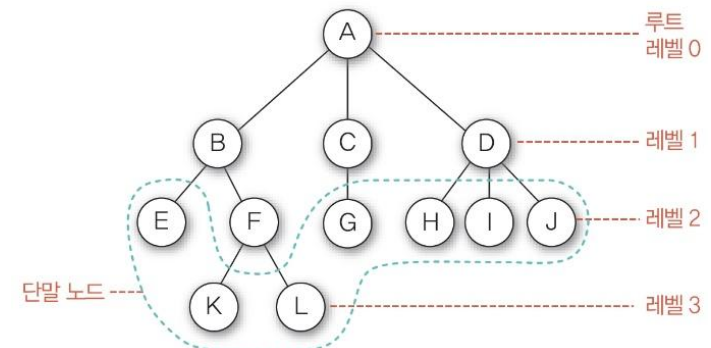


그림 7-2 트리의 구조와 구성 요소

1. 트리의 이해

- 포레스트(forest) : 서브 트리의 집합
 - ▶ 트리 A에서 노드 A를 제거하면, A의 자식 노드 B, C, D에 대한 서브 트리가 생기고, 이들의 집합은 포레스트가 됨

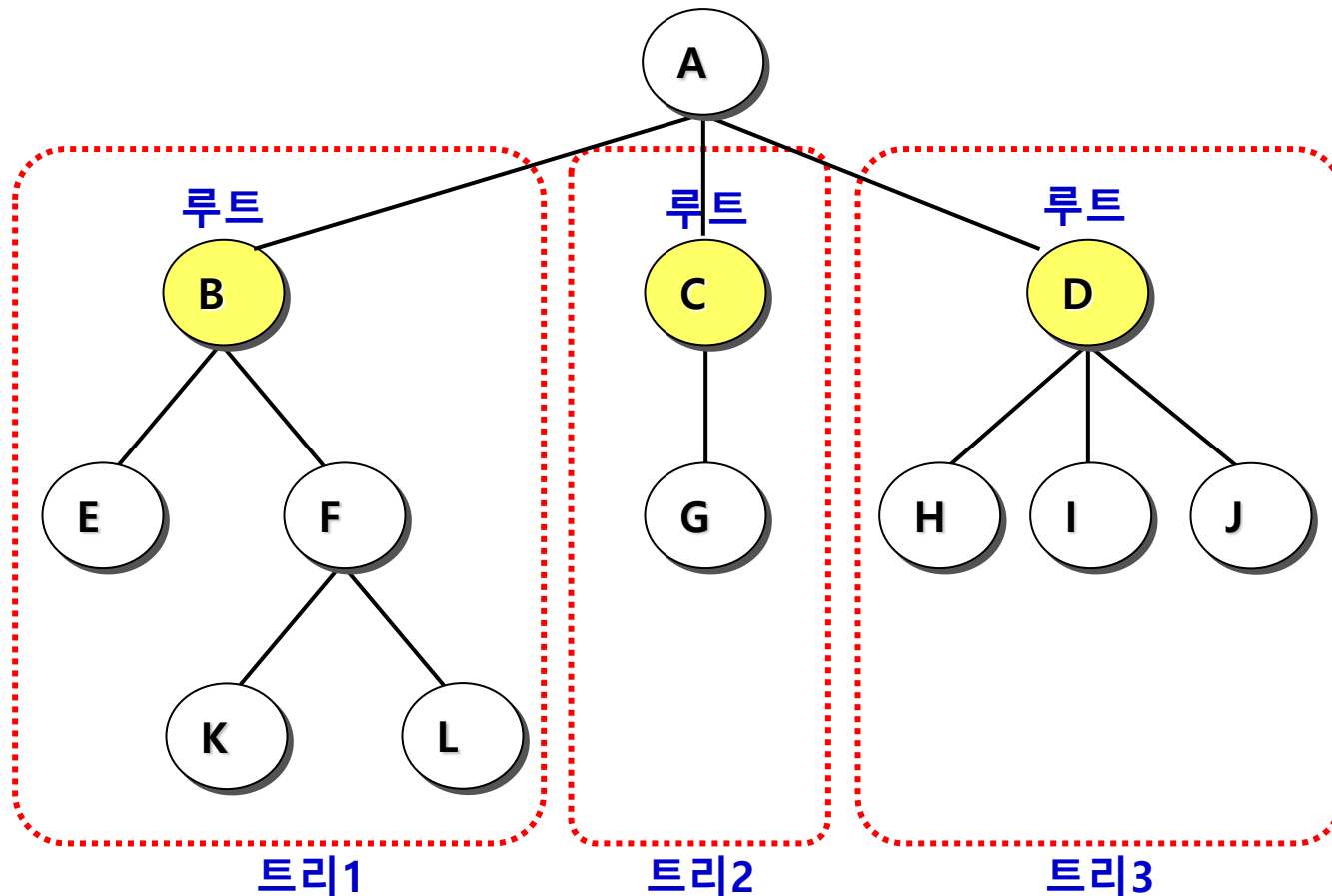


그림 7-3 [그림 7-2]에서 루트 노드 A를 제거하여 만든 포리스트

2. 이진 트리 : 개념과 구조

■ 이진 트리 Binary Tree

- 트리의 모든 노드의 차수를 2 이하로 제한하여 전체 트리의 차수가 2 이하가 되도록 정의 → **트리의 연산이 단순 명확해 진다.**
- 이진 트리의 모든 노드는 왼쪽 자식 노드와 오른쪽 자식 노드만 가짐
 - 부모 노드와 자식 노드 수와의 관계 ⇨ 1:2
 - 공백 노드도 자식 노드로 취급
 - $0 \leq \text{노드의 차수} \leq 2$

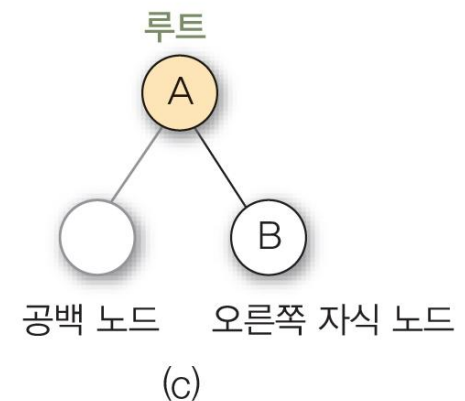
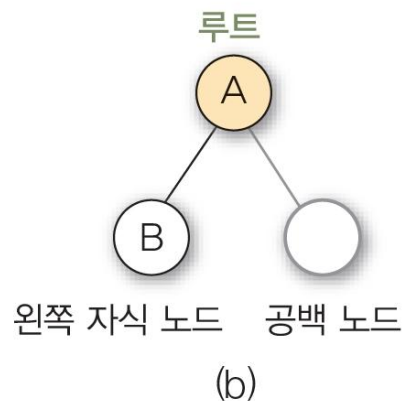
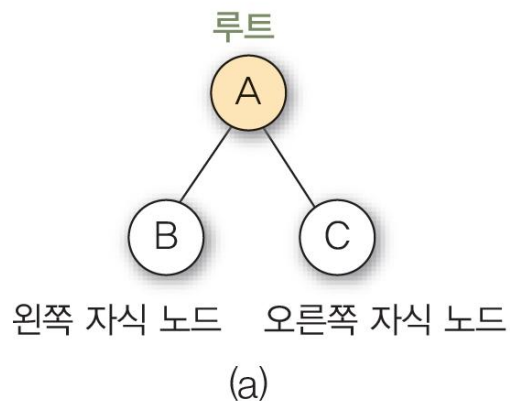


그림 7-4 이진 트리의 기본 구조

2. 이진 트리 : 개념과 구조

- 이진 트리는 순환적 구성 (똑같은 구조의 작은 트리가 계속 들어있는 구조)
 - 노드의 왼쪽 자식 노드를 루트로 하는 왼쪽 서브 트리도 이진 트리
 - 노드의 오른쪽 자식 노드를 루트로 하는 오른쪽 서브 트리도 이진 트리

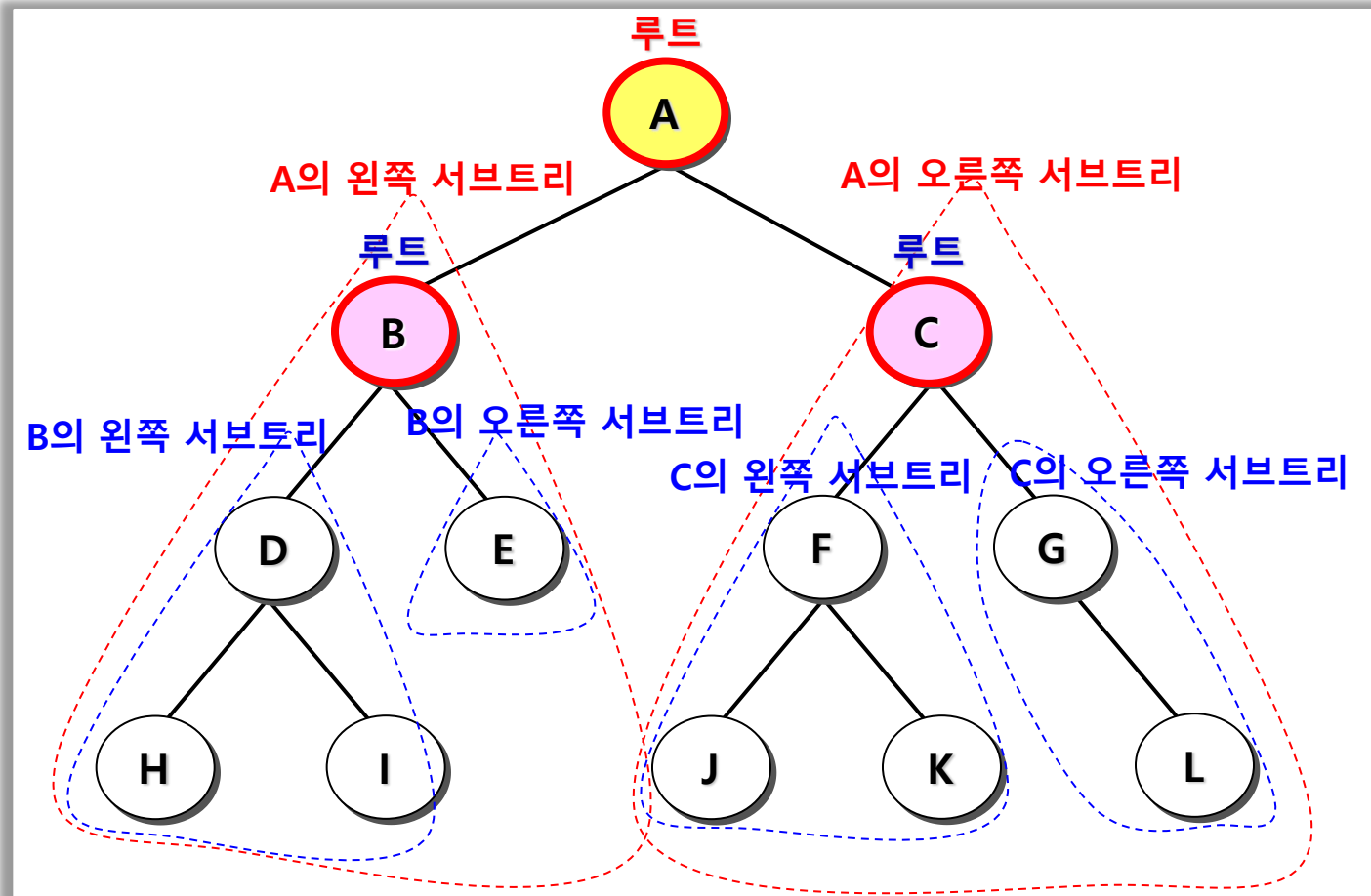
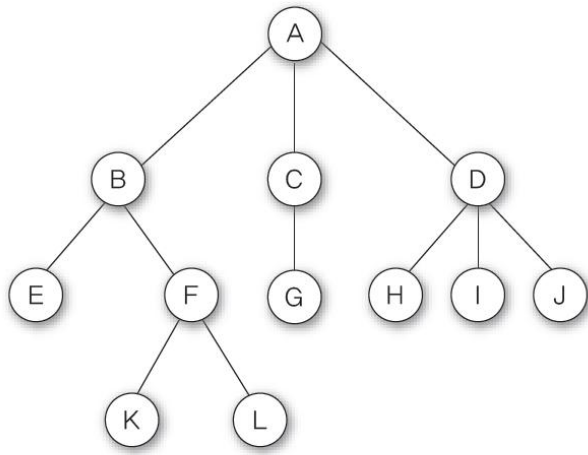


그림 7-5 이진 트리의 서브 트리

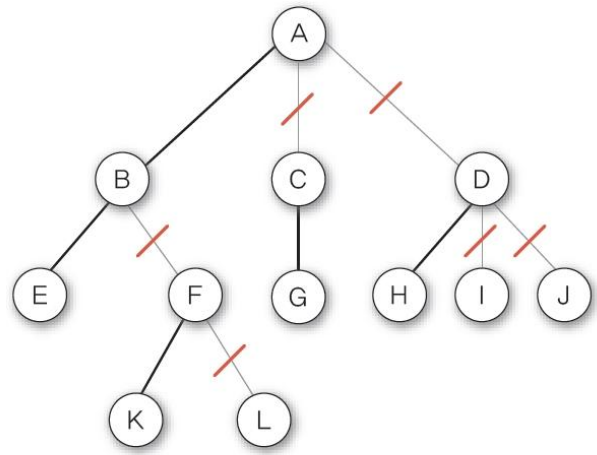
2. 이진 트리 : 개념과 구조

- 일반 트리를 이진 트리로 변환

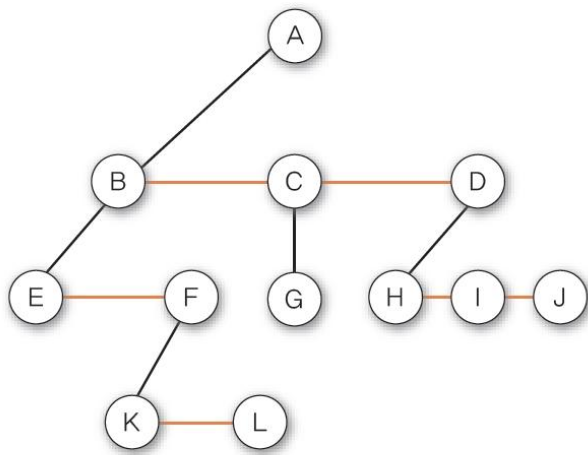
① 일반 트리 A



① 첫 번째 자식 노드 간선만 남기고 나머지 간선 제거



② 형제 노드를 간선으로 연결



③ 시계 방향으로 45° 회전

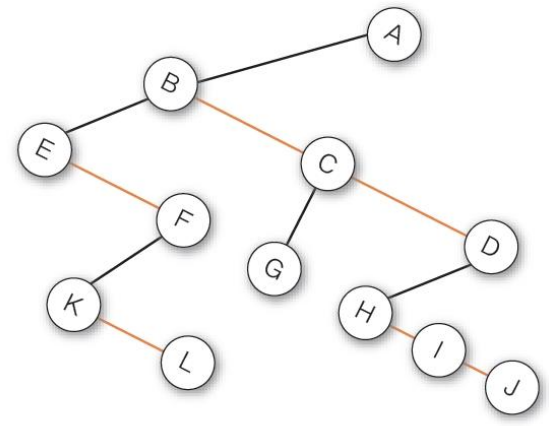


그림 7-6 일반 트리를 이진 트리로 변환하는 과정

ADT 7-1 이진 트리의 추상 자료형

ADT BinaryTree

데이터 : 공백이거나 루트 노드, 왼쪽 서브 트리, 오른쪽 서브 트리로 구성된 노드들의 유한 집합

연산 :

$bt, bt1, bt2 \in \text{BinaryTree}; \text{item} \in \text{Element};$

// 공백 이진 트리를 생성하는 연산

$\text{createBT}() ::= \text{create an empty binary tree};$

// 이진 트리가 공백인지 확인하는 연산

$\text{isEmpty}(bt) ::= \text{if } (bt \text{ is empty}) \text{ then return true}$
 $\quad \quad \quad \text{else return false};$

// 두 개의 이진 서브 트리를 연결하여 하나로 만드는 연산

$\text{makeBT}(\text{item}, bt1, bt2) ::= \text{return } \{\text{item을 루트로 하고}$
 $\quad \quad \quad \text{bt1을 왼쪽 서브 트리, bt2를 오른쪽 서브 트리로 하는 이진 트리}\}$

// 이진 트리의 왼쪽 서브 트리를 구하는 연산

$\text{leftSubtree}(bt) ::= \text{if } (\text{isEmpty}(bt)) \text{ then return NULL}$
 $\quad \quad \quad \text{else return left subtree of } bt;$

// 이진 트리의 오른쪽 서브 트리를 구하는 연산

$\text{rightSubtree}(bt) ::= \text{if } (\text{isEmpty}(bt)) \text{ then return NULL}$
 $\quad \quad \quad \text{else return right subtree of } bt;$

// 이진 트리에서 루트 노드의 데이터(item)를 구하는 연산

$\text{data}(bt) ::= \text{if } (\text{isEmpty}(bt)) \text{ then return NULL}$
 $\quad \quad \quad \text{else return the item in the root node of } bt;$

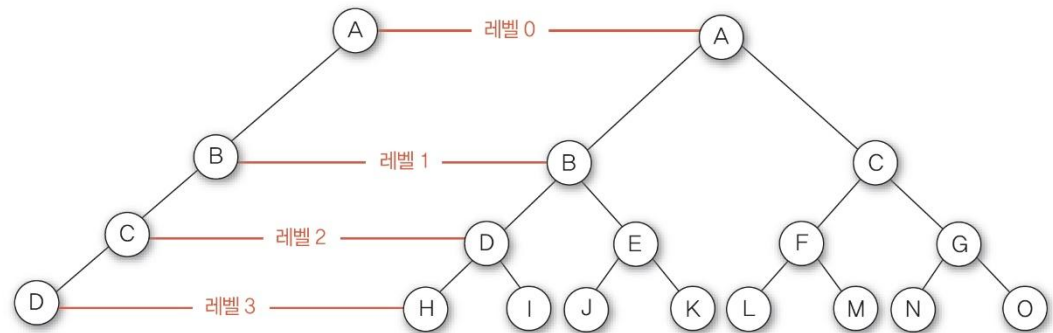
End BinaryTree

2. 이진 트리 : 추상 자료형

– 이진 트리의 특성

- 정의1) 노드가 n 개인 이진 트리는 항상 간선이 $(n-1)$ 개임
 - 루트를 제외한 $(n-1)$ 개의 노드가 부모 노드와 연결되는 한 개의 간선을 가짐
- 정의2) 높이가 h 인 이진 트리가 가질 수 있는 노드 개수는 최소 $(h+1)$ 개이며, 최대 $(2^{h+1}-1)$ 개
 - 이진 트리의 높이가 h 가 되려면 한 레벨에 최소한 한 개의 노드가 있어야 하므로 높이가 h 인 이진 트리의 최소 노드의 개수는 $(h+1)$ 개
 - 하나의 노드는 최대 2개의 자식 노드를 가질 수 있으므로 레벨 i 에서의 노드의 최대 개수는 2^i 개 이므로 높이가 h 인 이진 트리 전체의 노드 개수는

$$\sum_{i=0}^h 2^i = 2^{h+1} - 1 \text{ 개}$$



(a) 최소 노드를 갖는 이진 트리

(b) 최대 노드를 갖는 이진 트리

그림 7-7 높이가 3이면서 최소 노드를 갖는 이진 트리와 최대 노드를 갖는 이진 트리

2. 이진 트리 : 종류

■ 이진 트리의 종류

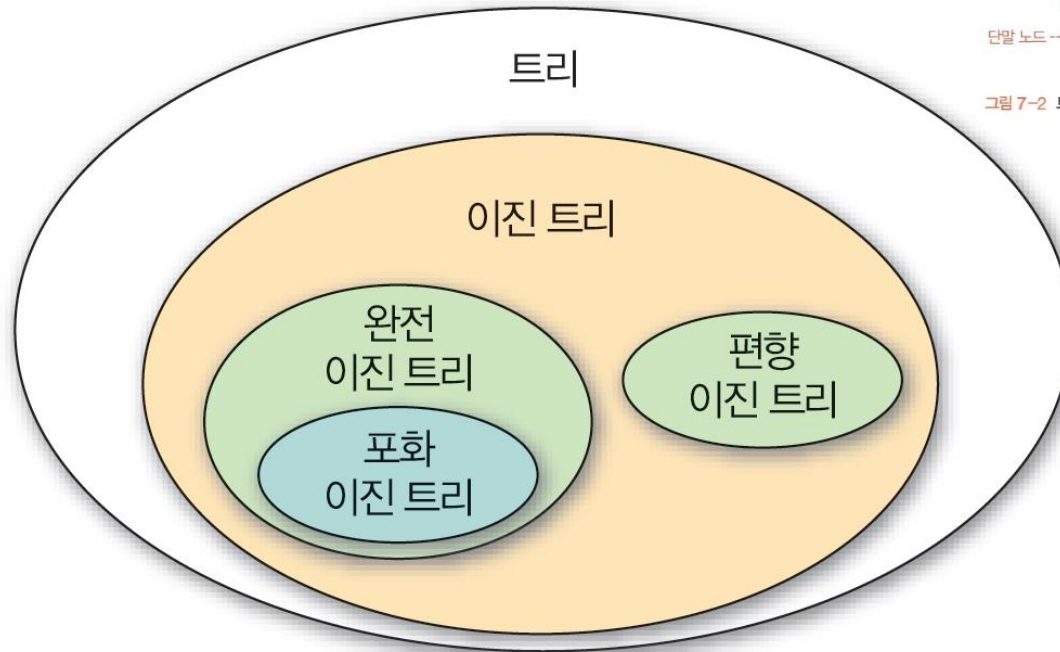


그림 7-8 이진 트리의 종류

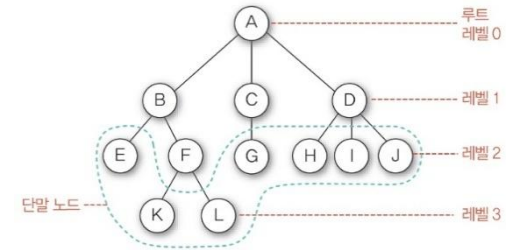


그림 7-2 트리의 구조와 구성 요소

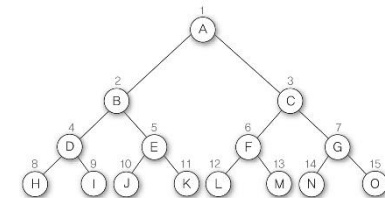


그림 7-9 높이가 3인 포화 이진 트리

2. 이진 트리 : 종류

■ 이진 트리의 종류

– 포화 이진 트리 Full Binary Tree

- 모든 레벨에 노드가 포화상태로 차 있는 이진 트리
- 높이가 h 일 때, 최대의 노드 개수인 $(2^{h+1}-1)$ 의 노드를 가진 이진 트리
- 루트를 1번으로 하여 $2^{h+1}-1$ 까지 정해진 위치에 대한 노드 번호를 가짐

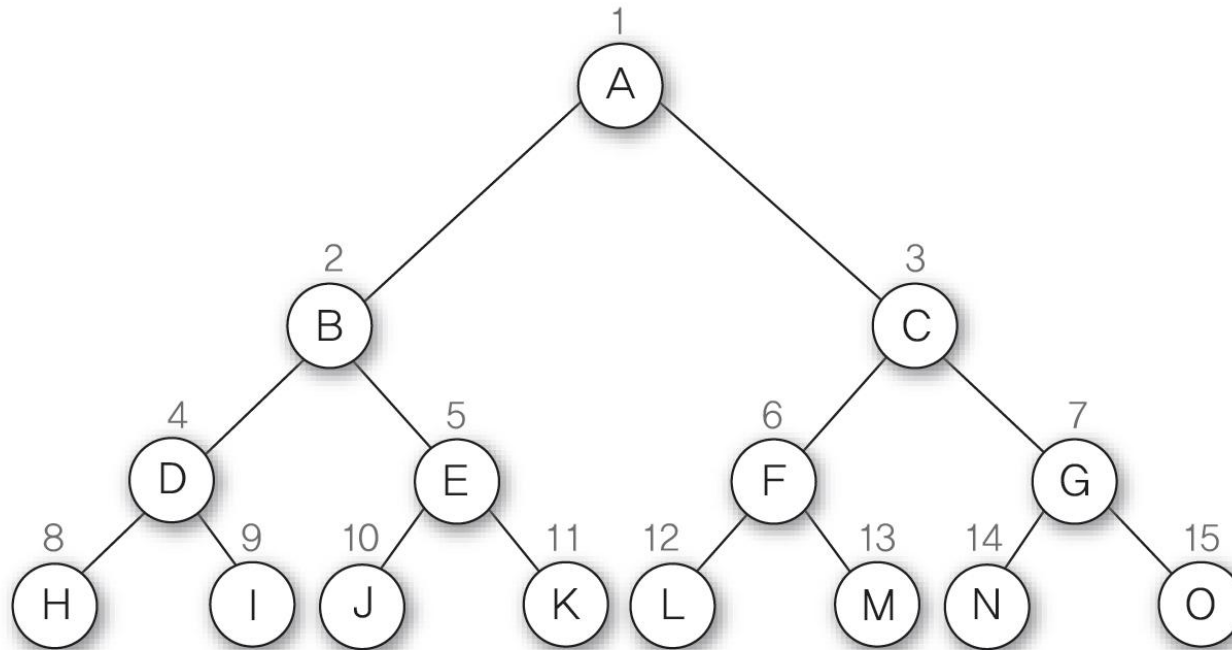


그림 7-9 높이가 3인 포화 이진 트리

2. 이진 트리 : 종류

– 완전 이진 트리 | Complete Binary Tree

- 높이가 h 이고 노드 수가 n 개일 때 (단, $n < 2^{h+1}-1$), 노드 위치가 포화 이진 트리에서의 노드 1번부터 n 번까지의 위치와 완전히 일치하는 이진 트리
- 완전 이진 트리에서는 $(n+1)$ 번부터 $(2^{h+1}-1)$ 번까지 노드는 모두 공백 노드

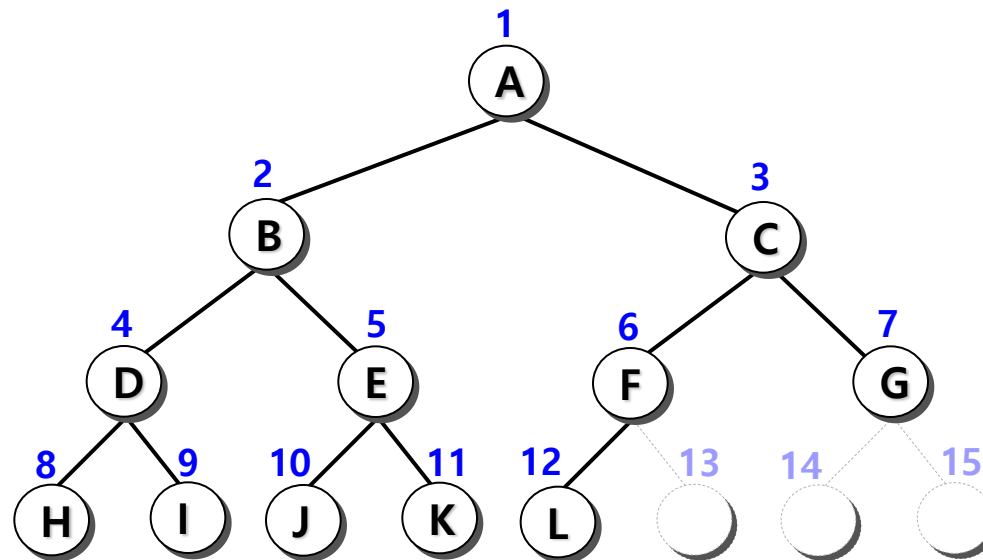
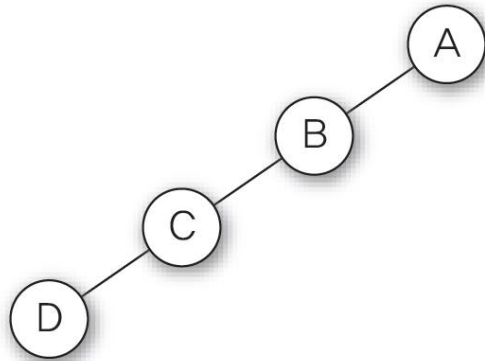


그림 7-10 높이가 3인 완전 이진 트리

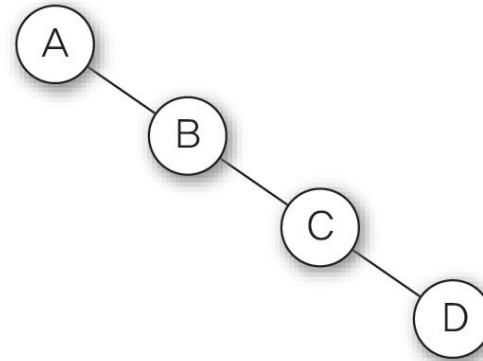
2. 이진 트리 : 종류

– 편향 이진 트리 | Skewed Binary Tree

- 높이가 h 일 때 $h+1$ 개의 노드를 가지면서 모든 노드가 왼쪽이나 오른쪽 중 한 방향으로만 서브 트리를 가지고 있는 트리



(a) 왼쪽 편향 이진 트리



(b) 오른쪽 편향 이진 트리

그림 7-11 높이가 3인 편향 이진 트리

3. 이진 트리 구현 : 순차 자료구조를 이용한 구현

- 순차 자료구조를 이용한 이진트리의 구현
 - 1차원 배열의 순차 자료구조 사용
 - 높이가 h 인 포화 이진 트리의 노드번호를 배열의 인덱스로 사용
 - 인덱스 0번 : 실제로 사용하지 않고 비워 둠.
 - 인덱스 1번 : 루트 저장

3. 이진 트리 구현 : 순차 자료구조를 이용한 구현

- 완전 이진 트리의 1차원 배열 표현

표 7-1 이진 트리를 표현한 1차원 배열에서의 인덱스 관계

노드	인덱스	성립 조건
노드 i의 부모 노드	$\lfloor i/2 \rfloor$	$i > 1$
노드 i의 왼쪽 자식 노드	$2 \times i$	$(2 \times i) \leq n$
노드 i의 오른쪽 자식 노드	$(2 \times i) + 1$	$(2 \times i + 1) \leq n$
루트 노드	1	$n > 0$

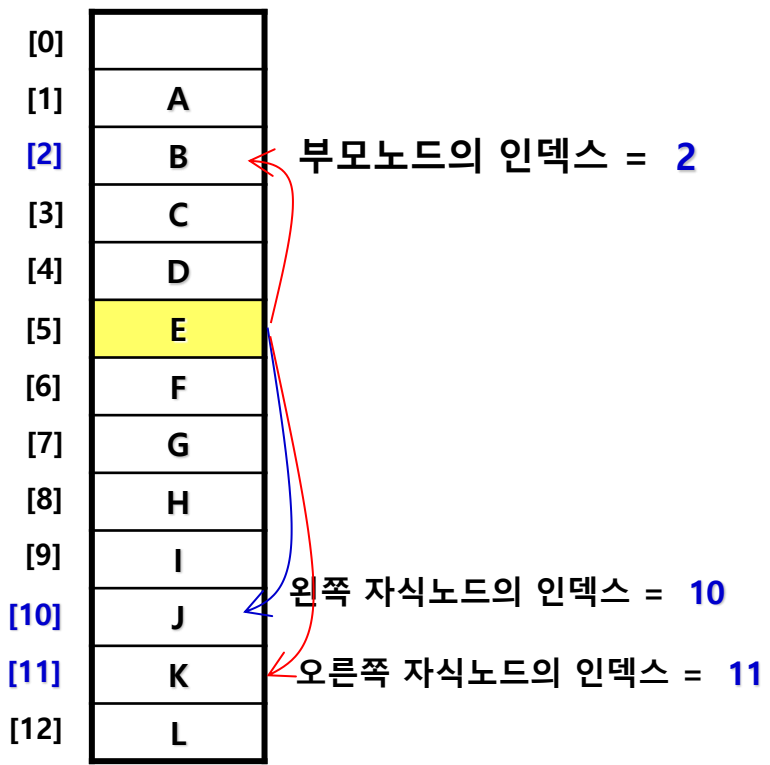
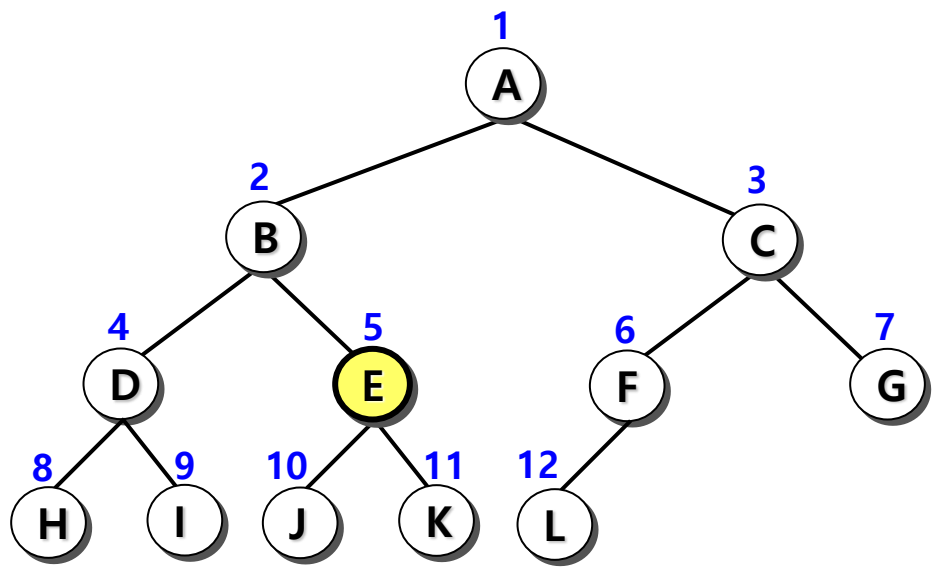
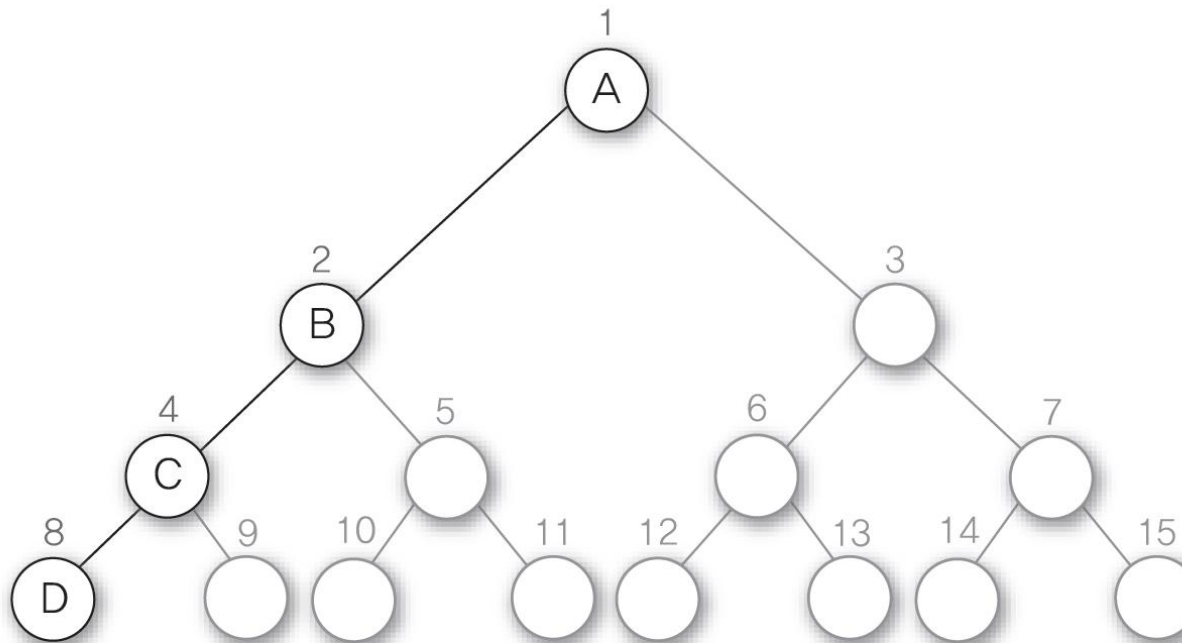


그림 7-12 이진 트리의 배열 표현 예 1 : 완전 이진 트리의 1차원 배열 표현

3. 이진 트리 구현 : 순차 자료구조를 이용한 구현

- 편향 이진 트리의 1차원 배열 표현



[0]	
[1]	A
[2]	B
[3]	
[4]	C
[5]	
[6]	
[7]	
[8]	D

그림 7-13 이진 트리의 배열 표현 예 2 : 편향 이진 트리의 1차원 배열 표현

3. 이진 트리 구현 : 연결 자료구조를 이용한 구현

- 연결 자료구조를 이용한 이진트리의 구현
 - 포인터를 사용하여 이진트리 구현
 - 데이터를 저장하는 데이터 필드, 왼쪽 자식 노드를 연결하는 왼쪽 링크 필드, 오른쪽 자식 노드를 연결하는 오른쪽 링크 필드로 구성. 자식 노드가 없으면 링크 필드에 NULL을 저장하여 NULL 포인터로 설정



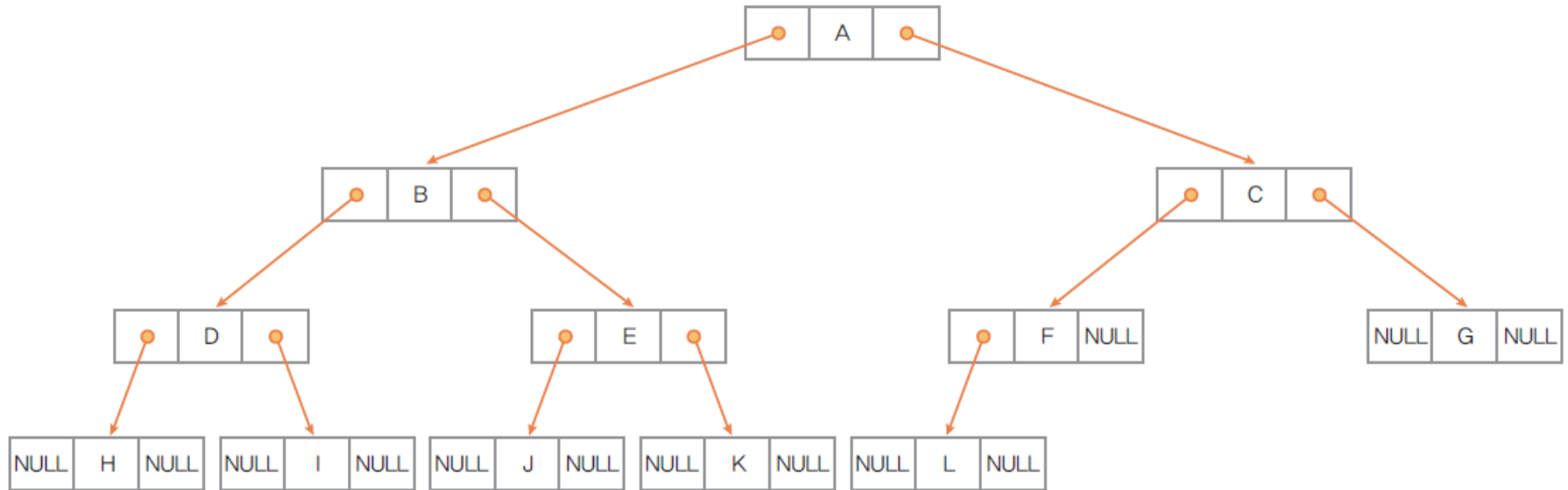
그림 7-14 이진 트리 노드의 구조

```
typedef struct treeNode {
    char data;
    struct treeNode *left;
    struct treeNode *right;
} treeNode;
```

그림 7-15 이진 트리 노드의 C 구조체 정의

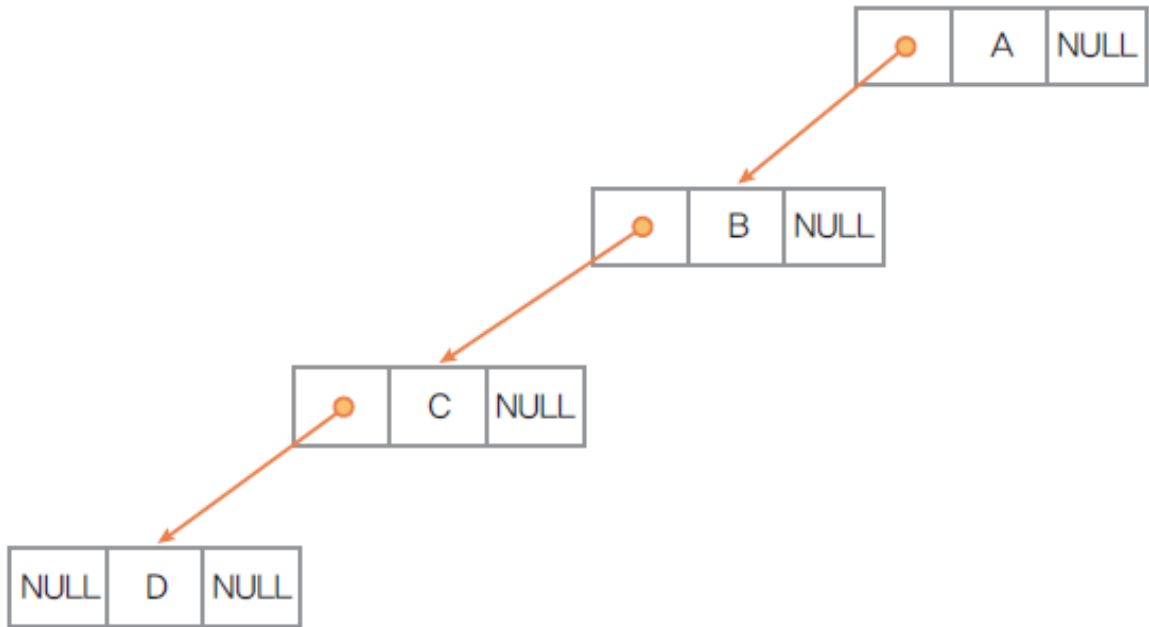
3. 이진 트리 구현 : 연결 자료구조를 이용한 구현

- 완전 이진 트리와 편향 이진 트리를 연결 자료구조 형태로 표현



(a) [그림 7-12] 완전 이진 트리에 대한 연결 자료구조 표현

3. 이진 트리 구현 : 연결 자료구조를 이용한 구현



(b) [그림 7-13] 편향 이진 트리에 대한 연결 자료구조 표현

그림 7-16 이진 트리의 연결 자료구조 표현

4. 이진 트리의 순회

■ 이진 트리 순회(traversal)의 개념

– 모든 원소를 빠트리거나 중복하지 않고 처리하는 연산

- 작업 D : 현재 노드를 방문하여 처리한다. **Data**
- 작업 L : 현재 노드의 왼쪽 서브 트리로 이동한다. **Left**
- 작업 R : 현재 노드의 오른쪽 서브 트리로 이동한다. **Right**

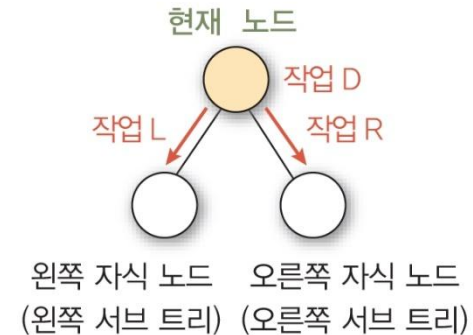
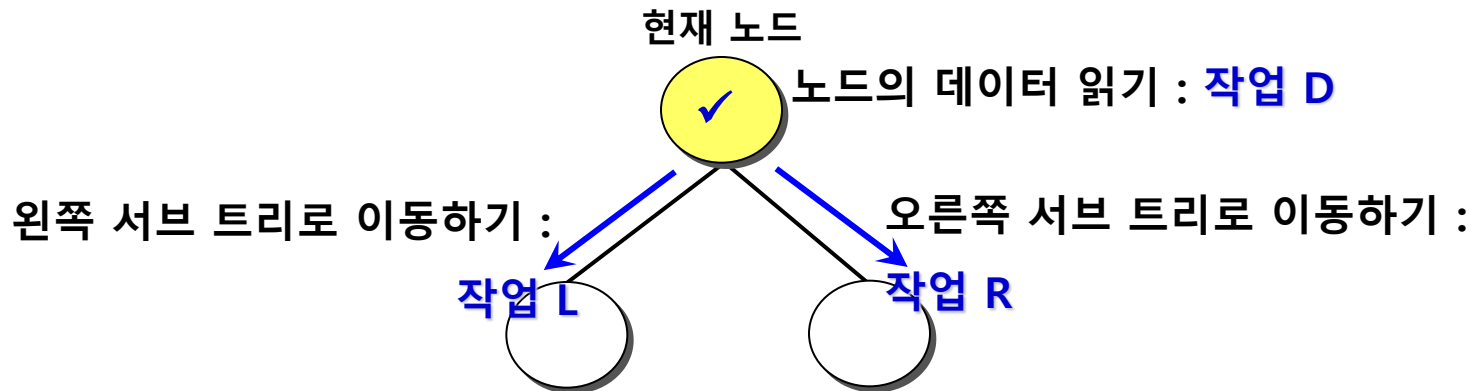


그림 7-17 이진 트리의 순회를 위한 세부 작업



4. 이진 트리의 순회

- 이진 트리가 순환적으로 정의되어 구성되어있으므로, 순회 작업도 서브 트리에 대해서 순환적으로 반복하여 완성한다.
- 왼쪽 서브 트리에 대한 순회를 오른쪽 서브 트리보다 먼저 수행한다.
- 순회의 종류
 - 전위 순회
 - 중위 순회
 - 후위 순회

4. 이진 트리의 순회

■ 전위 순회|preorder traversal

- $D \rightarrow L \rightarrow R$ 순서로, 현재 노드를 방문하여 처리하는 작업 D를 가장 먼저 수행

- 1 작업 D : 현재 노드 n을 처리한다.
- 2 작업 L : 현재 노드 n의 왼쪽 서브 트리로 이동한다. (서브 트리를 순회한다.)
- 3 작업 R : 현재 노드 n의 오른쪽 서브 트리로 이동한다. (서브 트리를 순회한다.)

그림 7-18 이진 트리의 전위 순회 작업 순서

알고리즘 7-1 이진 트리의 전위 순회

```
preorder(T)
  if (T ≠ NULL) then {
    visit T.data;
    preorder(T.left);
    preorder(T.right);
  }
end preorder()
```

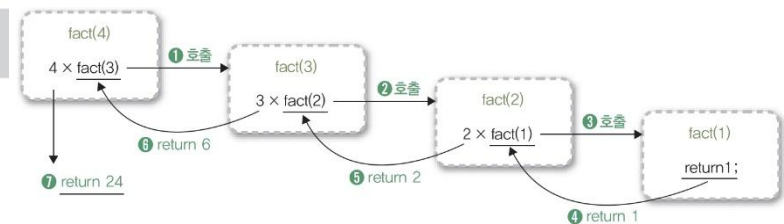


그림 2-45 $n=4$ 일 때 재귀호출과 반환

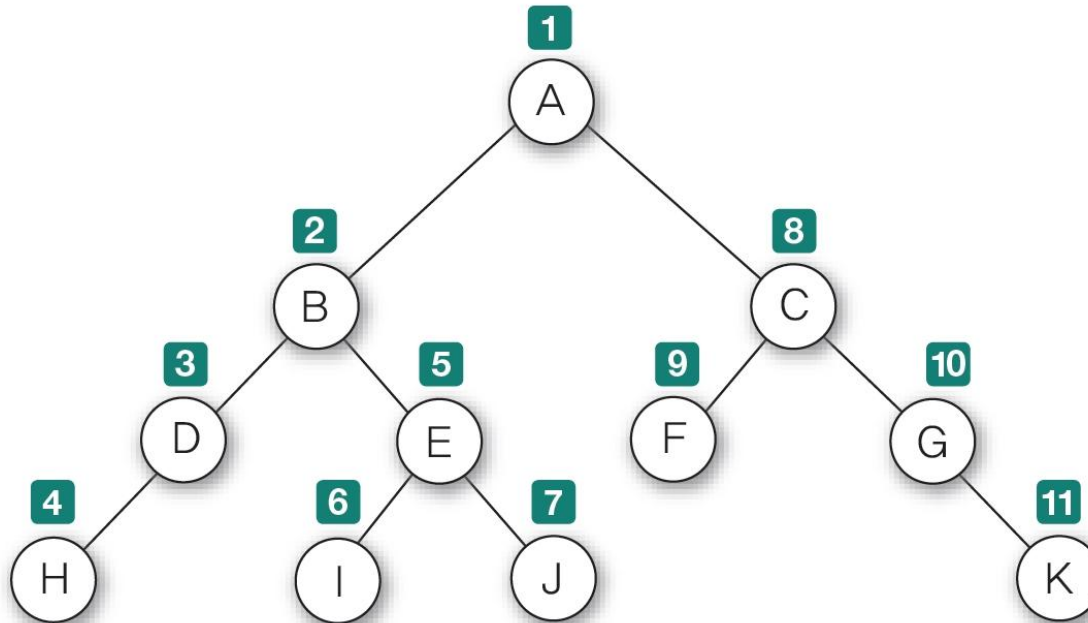


그림 2-42 재귀호출 개념을 이해하기 위한 마트료시카 인형

한 번에 완성할 수 있는 제일 작은 인형
→ 백이스케이스

4. 이진 트리의 순회

- 전위 순회의 예

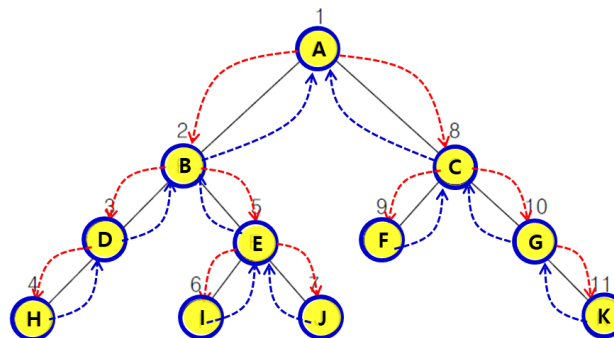


알고리즘 7-1 이진 트리의 전위 순회

```

preorder(T)
  if (T≠NULL) then {
    visit T.data;
    preorder(T.left);
    preorder(T.right);
  }
end preorder()
  
```

그림 7-19 이진 트리의 전위 순회 경로 : A-B-D-H-E-I-J-C-F-G-K



4. 이진 트리의 순회

- 수식 $A*B-C/D$ 를 이진 트리로 구성
 - 수식에 대한 이진 트리를 전위 순회하면, 전위 표기식을 구할 수 있음

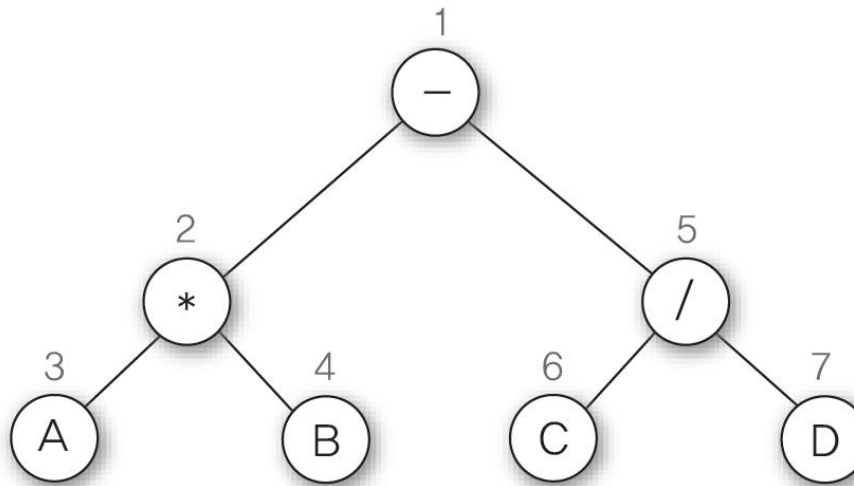


그림 7-20 수식에 대한 이진 트리의 전위 순회 경로 : $-*AB/CD$

4. 이진 트리의 순회

■ 중위 순회 | inorder traversal

- $L \rightarrow D \rightarrow R$ 순서로, 현재 노드를 방문하는 작업 D를 작업 L과 작업 R의 중간에 수행

- ① 작업 L : 현재 노드 n의 왼쪽 서브 트리로 이동한다. (서브 트리를 순회한다.)
- ② 작업 D : 현재 노드 n을 처리한다.
- ③ 작업 R : 현재 노드 n의 오른쪽 서브 트리로 이동한다. (서브 트리를 순회한다.)

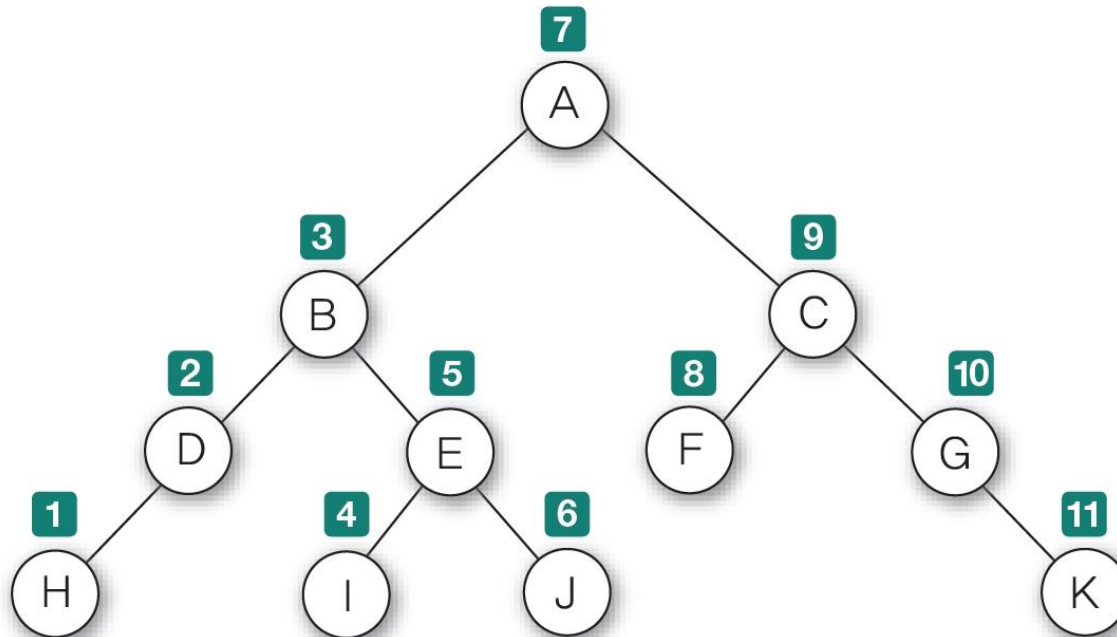
그림 7-21 이진 트리의 중위 순회 작업 순서

알고리즘 7-2 이진 트리의 중위 순회

```
inorder(T)
  if (T ≠ NULL) then {
    inorder(T.left);
    visit T.data;
    inorder(T.right);
  }
end inorder()
```

4. 이진 트리의 순회

- 중위 순회의 예



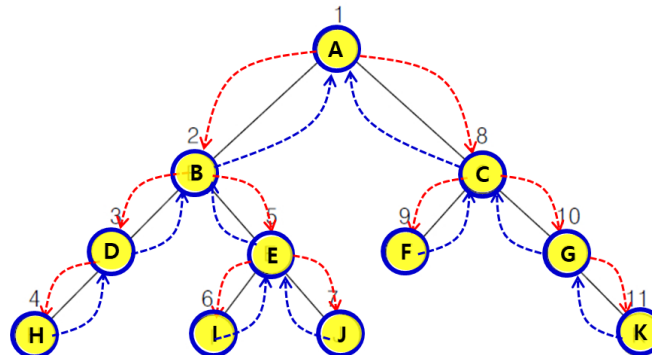
알고리즘 7-2 이진 트리의 중위 순회

```

inorder(T)
  if (T≠NULL) then {
    inorder(T.left);
    visit T.data;
    inorder(T.right);
  }
end inorder()

```

그림 7-22 이진 트리의 중위 순회 경로 : H-D-B-I-E-J-A-F-C-G-K



4. 이진 트리의 순회

- 수식 $A*B-C/D$ 를 이진 트리로 구성
 - 수식 이진 트리를 중위 순회하면, 수식에 대한 중위 표기식을 구할 수 있음

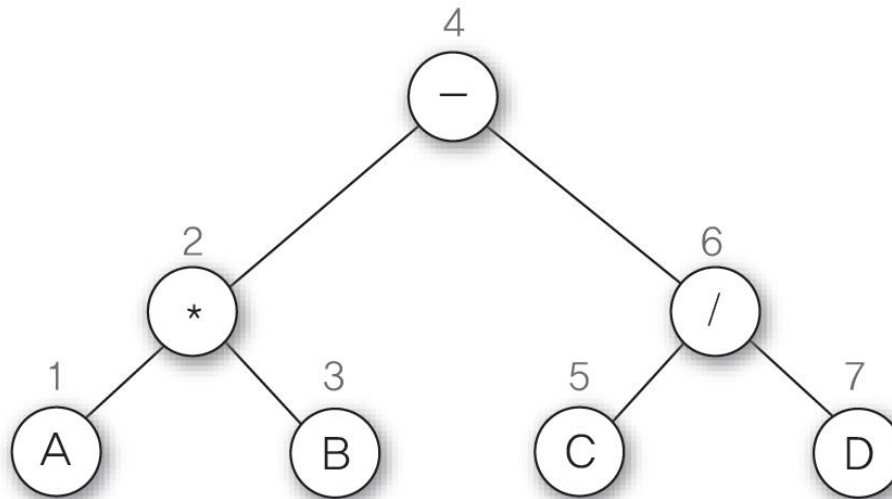


그림 7-23 수식에 대한 이진 트리의 중위 순회 경로 : $A*B-C/D$

4. 이진 트리의 순회

■ 후위 순회 | postorder traversal

– L-R-D 순서로 현재 노드를 방문하는 D 작업을 가장 나중에 수행

- ① 작업 L : 현재 노드 n의 왼쪽 서브 트리로 이동한다. (서브 트리를 순회한다.)
- ② 작업 R : 현재 노드 n의 오른쪽 서브 트리로 이동한다. (서브 트리를 순회한다.)
- ③ 작업 D : 현재 노드 n을 처리한다.

그림 7-24 이진 트리의 후위 순회 작업 순서

알고리즘 7-3 이진 트리의 후위 순회

```
postorder(T)
  if (T ≠ NULL) then {
    postorder(T.left);
    postorder(T.right);
    visit T.data;
  }
end postorder()
```

4. 이진 트리의 순회

- 후위 순회의 예

알고리즘 7-3 이진 트리의 후위 순회

```
postorder(T)
  if (T≠NULL) then {
    postorder(T.left);
    postorder(T.right);
    visit T.data;
  }
end postorder()
```

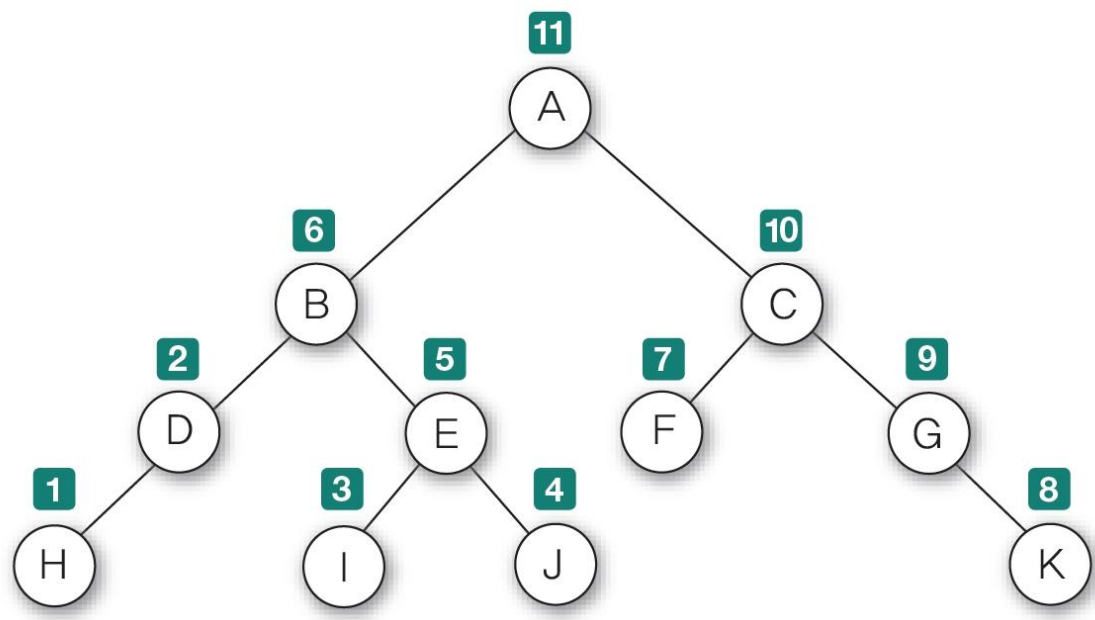
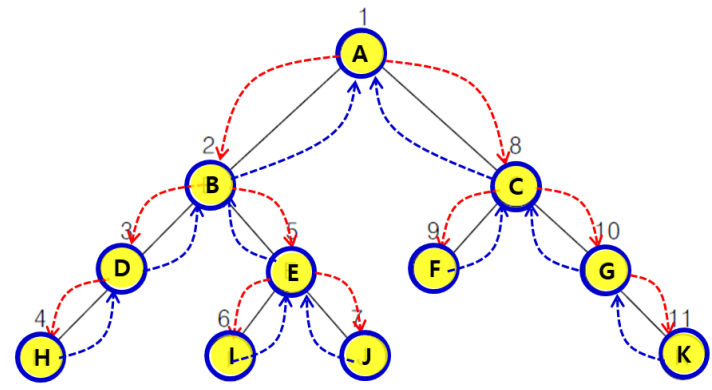


그림 7-25 이진 트리의 후위 순회 경로 : H-D-I-J-E-B-F-K-G-C-A



4. 이진 트리의 순회

- 수식 $A*B-C/D$ 를 이진 트리로 구성
 - 수식 이진 트리를 후위 순회하면, 수식에 대한 후위 표기식을 구할 수 있음

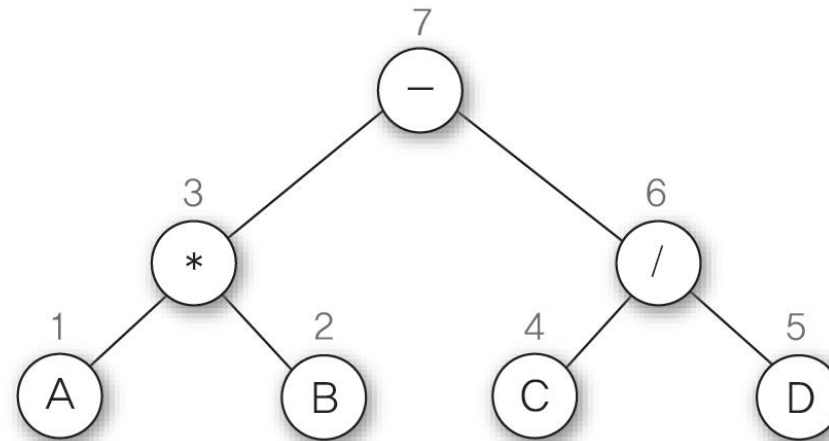


그림 7-26 수식에 대한 이진 트리의 후위 순회 경로 : $AB*CD/-$

4. 이진 트리의 순회

- [예제 7-1] 이진 트리 순회하기 : [교재 352p](#)
 - 수식 ($A*B-C/D$)에 대한 이진 트리를 연결 자료구조 방식으로 표현
 - makeRootNode() 함수를 이용해 전위 순회, 중위 순회, 후위 순회를 수행한 경로를 출력하여 확인하는 프로그램
- 실행 결과

```
preorder : -*AB/CD
inorder : A*B-C/D
postorder : AB*CD/-
```

4. 이진 트리의 순회

예제 7-1 이진 트리 순회하기

[traversalBT.h]

```

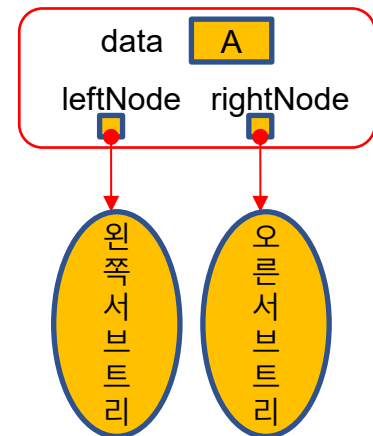
01  #pragma once
02  typedef char element;
03
04  typedef struct treeNode {    // 연결 자료구조로 구성하기 위해 트리의 노드 정의
05      element data;
06      struct treeNode* left;   // 왼쪽 서브 트리에 대한 링크 필드
07      struct treeNode* right; // 오른쪽 서브 트리에 대한 링크 필드
08  } treeNode;
09
10  treeNode* makeRootNode(element data, treeNode* leftNode, treeNode* rightNode);
11  void preorder(treeNode* root);
12  void inorder(treeNode* root);
13  void postorder(treeNode* root);
    
```

[traversalBT.c]

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  #include "traversalBT.h"
04
05  // data를 루트 노드로 하여 왼쪽 서브 트리와 오른쪽 서브 트리를 연결하는 연산
06  treeNode* makeRootNode(element data, treeNode* leftNode, treeNode* rightNode) {
07      treeNode* root = (treeNode*)malloc(sizeof(treeNode));
08      root->data = data;
09      root->left = leftNode;
10      root->right = rightNode;
11      return root;
12  }
    
```

treeNode

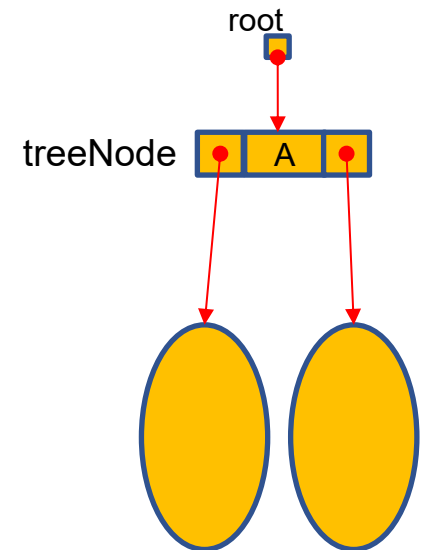


4. 이진 트리의 순회

```

13
14 // 이진 트리에 대한 전위 순회 연산 [알고리즘 7-1] 구현
15 void preorder(treeNode* root) {
16     if (root) {
17         printf("%c", root->data); //작업D
18         preorder(root->left);    //작업L
19         preorder(root->right);   //작업R
20     }
21 }
22
23 // 이진 트리에 대한 중위 순회 연산 [알고리즘 7-2] 구현
24 void inorder(treeNode* root) {
25     if (root) {
26         inorder(root->left);    //작업L
27         printf("%c", root->data); //작업D
28         inorder(root->right);   //작업R
29     }
30 }
31
32 // 이진 트리에 대한 후위 순회 연산 [알고리즘 7-3] 구현
33 void postorder(treeNode* root) {
34     if (root) {
35         postorder(root->left);   //작업L
36         postorder(root->right); //작업R
37         printf("%c", root->data); //작업D
38     }
39 }

```



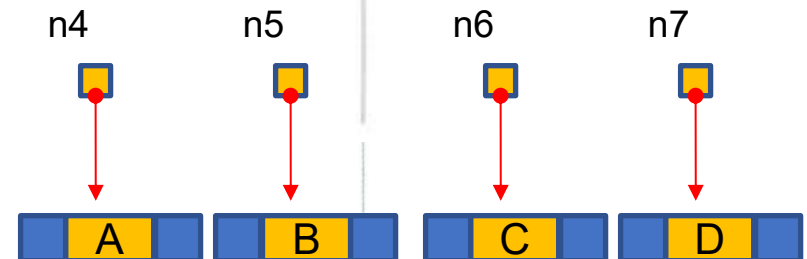
4. 이진 트리의 순회

[ex7_1.c]

```

01  #include <stdio.h>
02  #include "traversalBT.h"
03
04  int main(void) {
05      // (A*B-C/D) 수식 이진 트리 만들기
06      treeNode* n7 = makeRootNode('D', NULL, NULL);
07      treeNode* n6 = makeRootNode('C', NULL, NULL);
08      treeNode* n5 = makeRootNode('B', NULL, NULL);
09      treeNode* n4 = makeRootNode('A', NULL, NULL);
10
11      treeNode* n3 = makeRootNode('/', n6, n7);
12      treeNode* n2 = makeRootNode('*', n4, n5);
13      treeNode* n1 = makeRootNode('-', n2, n3);
14
15      printf("\n preorder : ");
16      preorder(n1);
17
18      printf("\n inorder : ");
19      inorder(n1);
20
21      printf("\n postorder : ");
22      postorder(n1);
23
24      getchar(); return 0;
25  }

```



```

preorder : - * AB / CD
inorder : A * B - C / D
postorder : AB * CD / -

```

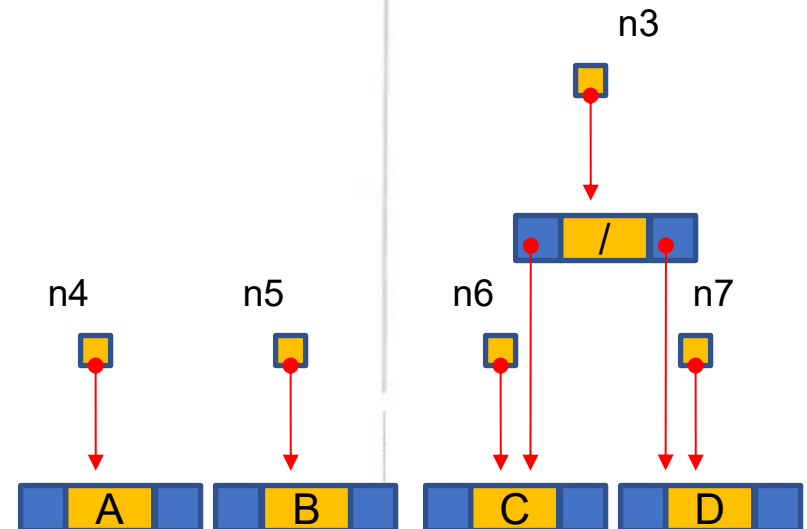

4. 이진 트리의 순회

[ex7_1.c]

```

01  #include <stdio.h>
02  #include "traversalBT.h"
03
04  int main(void) {
05      // (A*B-C/D) 수식 이진 트리 만들기
06      treeNode* n7 = makeRootNode('D', NULL, NULL);
07      treeNode* n6 = makeRootNode('C', NULL, NULL);
08      treeNode* n5 = makeRootNode('B', NULL, NULL);
09      treeNode* n4 = makeRootNode('A', NULL, NULL);
10
11      treeNode* n3 = makeRootNode('/', n6, n7);
12      treeNode* n2 = makeRootNode('*', n4, n5);
13      treeNode* n1 = makeRootNode('-', n2, n3);
14
15      printf("\n preorder : ");
16      preorder(n1);
17
18      printf("\n inorder : ");
19      inorder(n1);
20
21      printf("\n postorder : ");
22      postorder(n1);
23
24      getchar(); return 0;
25  }

```



```

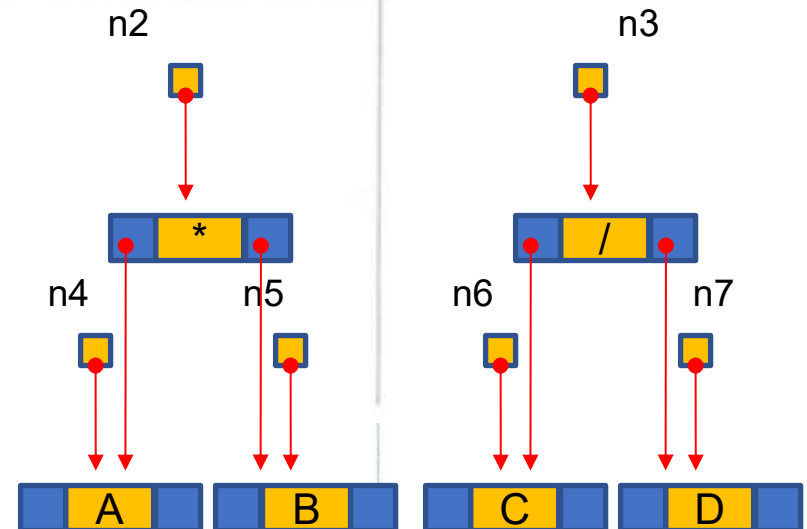
preorder : -*AB/CD
inorder : A*B-C/D
postorder : AB*CD/-

```


4. 이진 트리의 순회

[ex7_1.c]

```
01  #include <stdio.h>
02  #include "traversalBT.h"
03
04  int main(void) {
05      // (A*B-C/D) 수식 이진 트리 만들기
06      treeNode* n7 = makeRootNode('D', NULL, NULL);
07      treeNode* n6 = makeRootNode('C', NULL, NULL);
08      treeNode* n5 = makeRootNode('B', NULL, NULL);
09      treeNode* n4 = makeRootNode('A', NULL, NULL);
10
11      treeNode* n3 = makeRootNode('/', n6, n7);
12      treeNode* n2 = makeRootNode('*', n4, n5);
13      treeNode* n1 = makeRootNode('-', n2, n3);
14
15      printf("\n preorder : ");
16      preorder(n1);
17
18      printf("\n inorder : ");
19      inorder(n1);
20
21      printf("\n postorder : ");
22      postorder(n1);
23
24      getchar(); return 0;
25  }
```

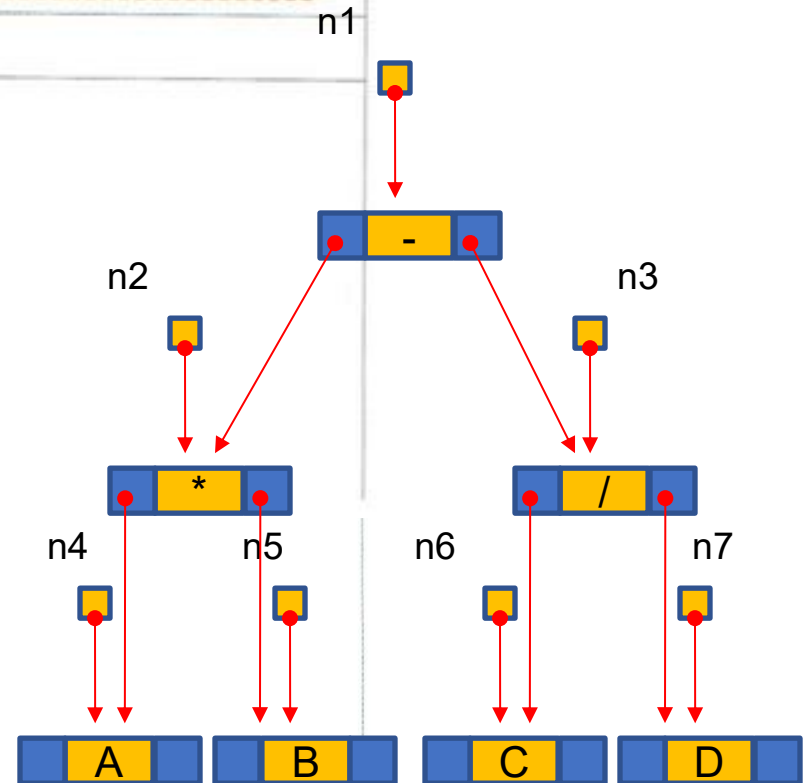


```
preorder : -*AB/CD
inorder : A*B-C/D
postorder : AB*CD/-
```

4. 이진 트리의 순회

[ex7_1.c]

```
01  #include <stdio.h>
02  #include "traversalBT.h"
03
04  int main(void) {
05      // (A*B-C/D) 수식 이진 트리 만들기
06      treeNode* n7 = makeRootNode('D', NULL, NULL);
07      treeNode* n6 = makeRootNode('C', NULL, NULL);
08      treeNode* n5 = makeRootNode('B', NULL, NULL);
09      treeNode* n4 = makeRootNode('A', NULL, NULL);
10
11      treeNode* n3 = makeRootNode('/', n6, n7);
12      treeNode* n2 = makeRootNode('*', n4, n5);
13      treeNode* n1 = makeRootNode('-', n2, n3);
14
15      printf("\n preorder : ");
16      preorder(n1);
17
18      printf("\n inorder : ");
19      inorder(n1);
20
21      printf("\n postorder : ");
22      postorder(n1);
23
24      getchar(); return 0;
25  }
```



```
preorder : -*AB/CD
inorder : A*B-C/D
postorder : AB*CD/-
```

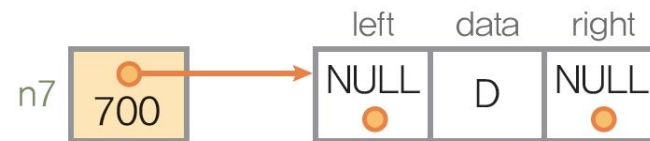
4. 이진 트리의 순회

[traversalBT.c]

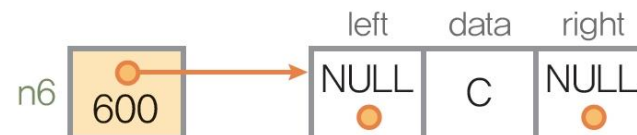
- 06~12행 : data를 루트 노드로 하여 왼쪽 서브 트리와 오른쪽 서브 트리를 연결하는 연산을 수행
- 15~21행 : 매개변수 root를 시작으로 하는 전위 순회 연산을 수행
- 24~30행 : 매개변수 root를 시작으로 하는 중위 순회 연산을 수행
- 33~39행 : 매개변수 root를 시작으로 하는 후위 순회 연산을 수행

[ex7_1.c]

- 06~12행 : $A*B-C/D$ 의 수식에 대한 이진 트리를 구성
 - 06행 : 데이터 필드가 'D'이고 왼쪽 서브 트리과 오른쪽 서브 트리가 공백 노드인 이진 트리 노드 n7을 선언

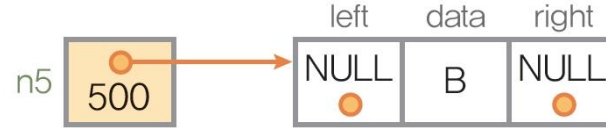


- 07행 : 데이터 필드가 'C'이고 왼쪽 서브 트리과 오른쪽 서브 트리가 공백 노드인 이진 트리 노드 n6 을 만듦

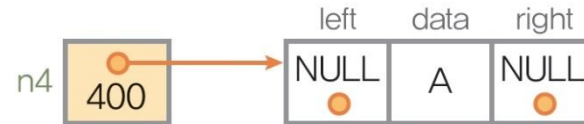


4. 이진 트리의 순회

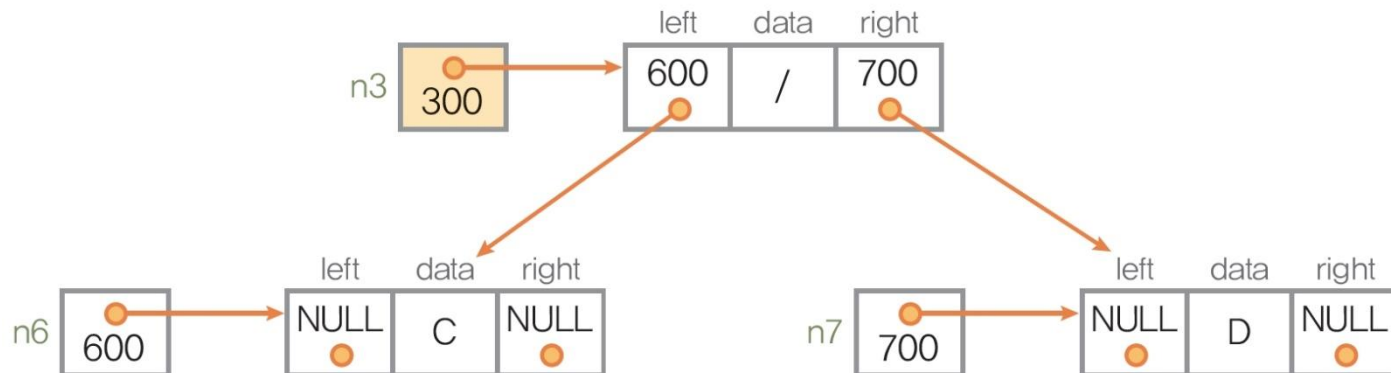
- 08행 : 데이터 필드가 B' '이고 왼쪽 서브 트리과 오른쪽 서브 트리가 공백 노드인 이진 트리 노드 n5 를 만듦



- 09행 : 데이터 필드가 A' '이고 왼쪽 서브 트리과 오른쪽 서브 트리가 공백 노드인 이진 트리 노드 n4 를 만듦

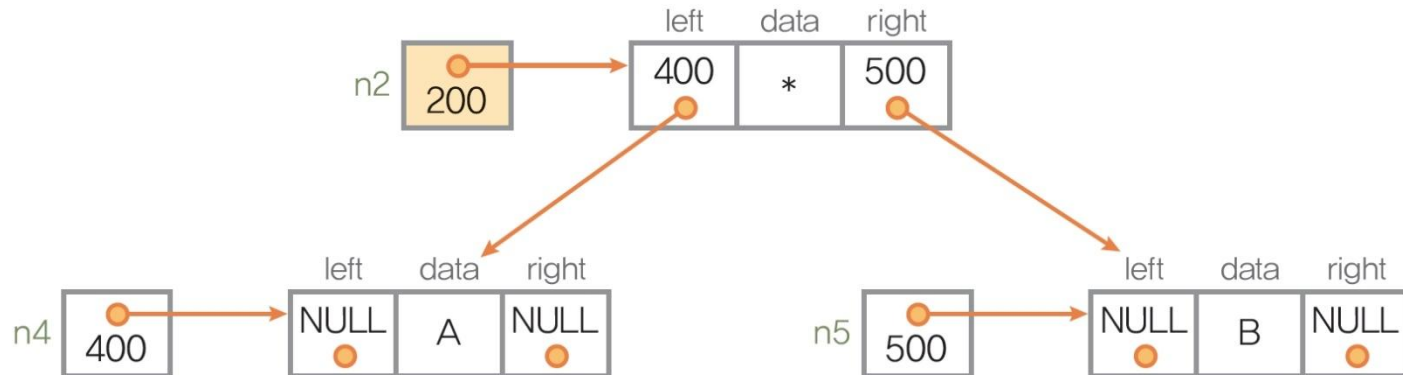


- 10행 : 데이터 필드가 '/'이고, 왼쪽 링크 필드에 노드 n6, 오른쪽 링크 필드에 노드 n7이 연결된 이진 트리 노드 n3을 만듦

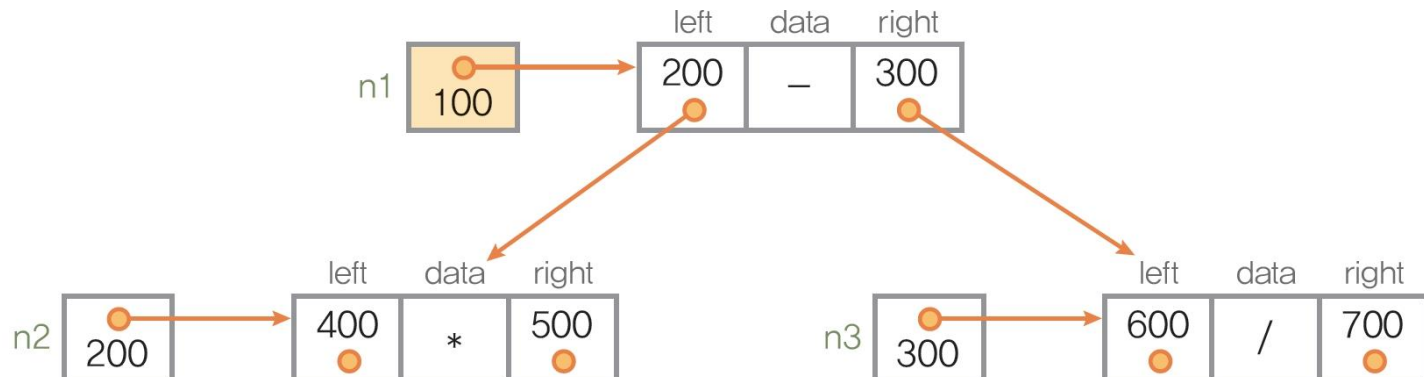


4. 이진 트리의 순회

- 11행 : 데이터 필드가 '*'이고 왼쪽 링크 필드에 노드 n4, 오른쪽 링크 필드에 노드 n5가 연결된 이진 트리 노드 n2를 만듦



- 12행 : 데이터 필드가 '-'이고 왼쪽 링크 필드에 노드 n2, 오른쪽 링크 필드에 노드 n3이 연결된 이진 트리 노드 n1을 만듦



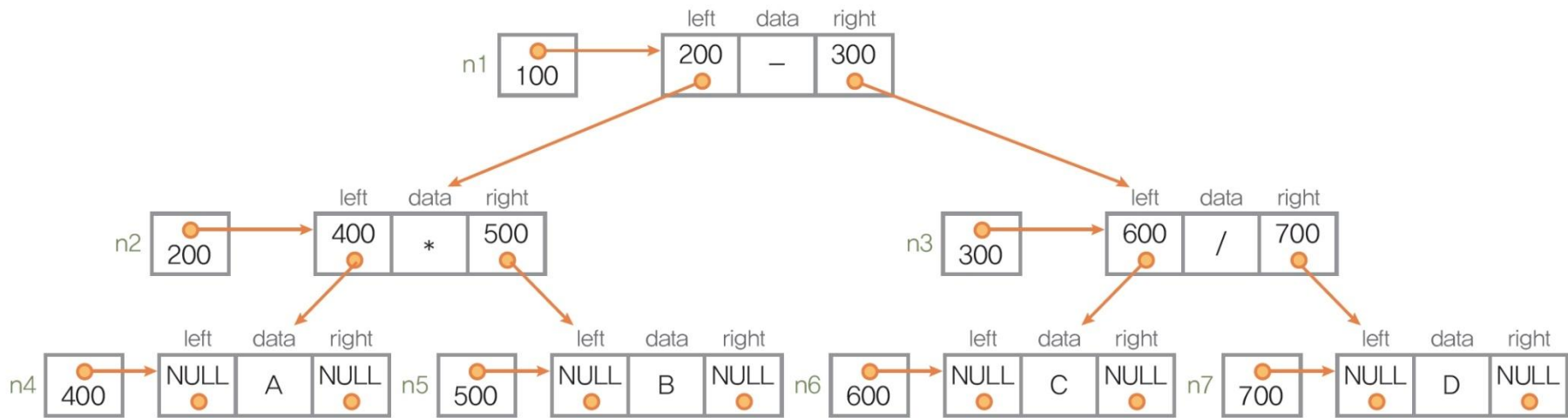
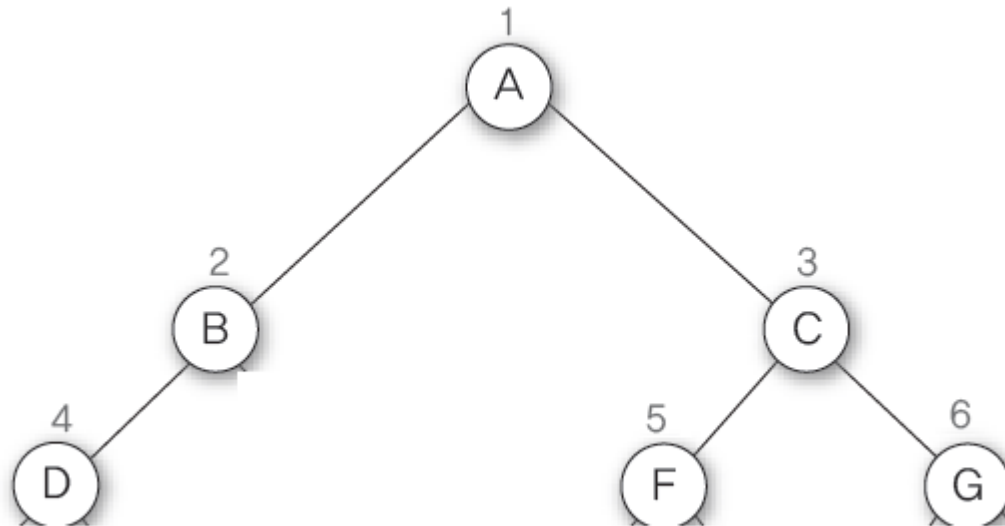


그림 7-27 makeRootNode()를 이용한 이진 트리의 구성 과정

4. 이진 트리의 순회

■ 이진트리 구성 연습

- 1. 아래 그림을 중위 순회 한 결과를 예상하시오.
- 2. 예제 7-1 을 수정하여 아래 그림의 이진 트리를 구성하고 중위 순회한 결과를 화면에 출력하시오.
- 위의 1, 2번 결과가 같은지 확인하시오.



4. 이진 트리의 순회

■ 이진 트리 순회의 응용 프로그램

- 컴퓨터의 폴더 구조가 이진 트리 구조인 경우 각 폴더를 순회하면서 용량을 계산하면 내 컴퓨터의 전체 용량이 계산 가능함.
- 폴더 전체 용량은 폴더에 저장된 파일 용량과 하위 폴더의 용량을 합한 값.
상위 폴더 용량을 계산하려면 먼저 하위 폴더 용량을 계산해야 하므로 후위 순회를 사용

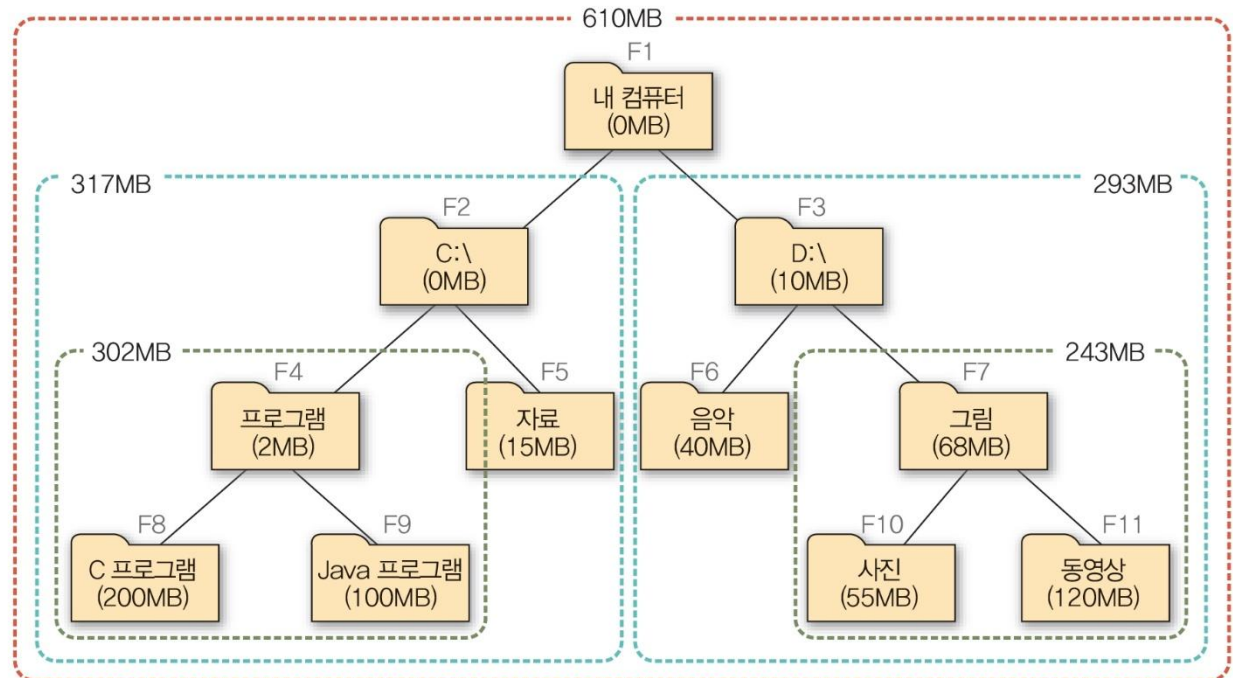


그림 7-28 후위 순회의 사용 예 : 컴퓨터 폴더 구조에서 전체 용량 계산하기

4. 이진 트리의 순회

- [프로그램] 전체 용량
 - = [프로그램] 용량 2MB + 하위 폴더 용량([C 프로그램] 용량 200MB+ [Java 프로그램] 용량 100MB)
= 2MB+(200MB+100MB) = 302MB
- [C:\] 전체 용량
 - = [C:\] 용량 0MB + 하위 폴더 용량([프로그램] 전체 용량 302MB + [자료] 용량 15MB)
= 0MB+(302MB+15MB) = 317MB
- [그림] 전체 용량
 - = [그림] 용량 68MB + 하위 폴더 용량([사진] 용량 55MB + [동영상] 용량 120MB)
= 68MB+(55MB+120MB) = 243MB
- [D:\] 전체 용량
 - = [D:\] 용량 10MB+ 하위 폴더 용량([음악] 용량 40MB + [그림] 전체 용량 243MB)
= 10MB+(40MB+243MB) = 293MB
- [내 컴퓨터] 전체 용량
 - = [내 컴퓨터] 용량 0MB + 하위 폴더 용량([C:\] 전체 용량 317MB + [D:\] 전체 용량 293MB)
= 0MB+(317MB+293MB) = 610MB

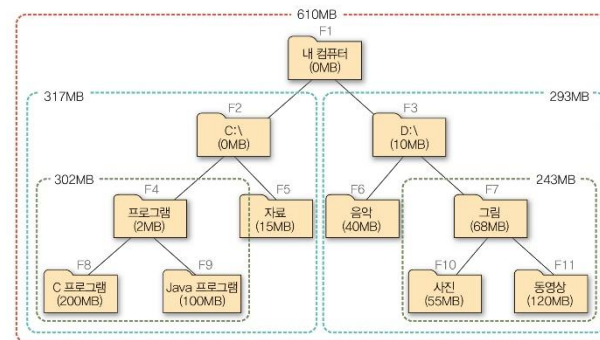
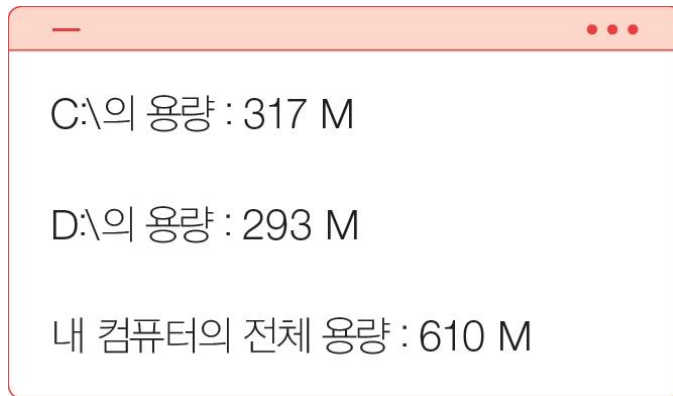


그림 7-28 후위 순회의 사용 예 : 컴퓨터 폴더 구조에서 전체 용량 계산하기

4. 이진 트리의 순회

- [예제 7-2] 이진 트리의 후위 순회를 이용해 폴더 용량 계산하기 : [교재 357p](#)
- 실행 결과



```

C:\의 용량 : 317 M

D:\의 용량 : 293 M

내 컴퓨터의 전체 용량 : 610 M
  
```

folderSize.h

```

1  || #pragma once
2  || int FolderSize;
3  || typedef struct treeNode { // 트리의 노드 구조 정의
4  ||     int size; // 데이터 필드
5  ||     struct treeNode* left; // 왼쪽 서브 트리에 대한 링크 필드
6  ||     struct treeNode* right; // 오른쪽 서브 트리에 대한 링크 필드
7  || } treeNode;
8
9  || treeNode* makeRootNode(int size, treeNode* leftNode, treeNode* rightNode);
10 || int postorder_FolderSize(treeNode* root);
  
```

4. 이진 트리의 순회

folderSize.c

```

1  <#include <stdlib.h>
2  <#include "folderSize.h"
3
4  // size를 루트 노드의 데이터 필드로 하여 왼쪽과 오른쪽 서브 트리를 연결하는 연산
5  <treeNode* makeRootNode(int size, treeNode* leftNode, treeNode* rightNode) {
6      treeNode* root = (treeNode*)malloc(sizeof(treeNode));
7      root->size = size;
8      root->left = leftNode;
9      root->right = rightNode;
10     return root;
11 }
12
13 // 각 폴더 크기를 계산하기 위한 후위 순회 연산
14 <int postorder_FolderSize(treeNode* root) {
15     if (root) {
16         postorder_FolderSize(root->left);
17         postorder_FolderSize(root->right);
18         FolderSize += root->size;
19     }
20     return FolderSize;
21 }

```

4. 이진 트리의 순회

ex7_2.c

```

1  √ #include <stdio.h>
2  | #include "folderSize.h"
3
4  √ int main(void) {
5  |     //이진 트리 생성하기
6  |     treeNode* F11 = makeRootNode(120, NULL, NULL);
7  |     treeNode* F10 = makeRootNode(55, NULL, NULL);
8  |     treeNode* F9 = makeRootNode(100, NULL, NULL);
9  |     treeNode* F8 = makeRootNode(200, NULL, NULL);
10 |     treeNode* F7 = makeRootNode(68, F10, F11);
11 |     treeNode* F6 = makeRootNode(40, NULL, NULL);
12 |     treeNode* F5 = makeRootNode(15, NULL, NULL);
13 |     treeNode* F4 = makeRootNode(2, F8, F9);
14 |     treeNode* F3 = makeRootNode(10, F6, F7);
15 |     treeNode* F2 = makeRootNode(0, F4, F5);
16 |     treeNode* F1 = makeRootNode(0, F2, F3);
17
18 |     FolderSize = 0;
19 |     printf("\n\n C:\\의 용량 : %d M \n", postorder_FolderSize(F2));
20 |     FolderSize = 0;
21 |     printf("\n\n D:\\의 용량 : %d M \n", postorder_FolderSize(F3));
22 |     FolderSize = 0;
23 |     printf("\n\n 내 컴퓨터의 전체 용량 : %d M \n", postorder_FolderSize(F1));
24
25 |     getchar(); return 0;
26 | }

```

C:\의 용량 : 317 M

D:\의 용량 : 293 M

내 컴퓨터의 전체 용량 : 610 M